Unified Resilience Testing Framework for ShaktiDB / DistDB

ShaktiDB (PostgreSQL) and DistDB

Document	Unified Resilience Testing Framework	
Version	1.0	
Date	28 July 2026	
Owner	Database Reliability Engineering	
Systems covered	ShaktiDB / standalone PostgreSQL; DistDB distributed cluster	
Scope	162 distinct scenarios across three tiers; 104 rated P0	
Classification	**CONFIDENTIAL — INTERNAL USE ONLY**	
Review cycle	Quarterly, or after any significant infrastructure or architecture change	

ShaktiDB

ShaktiDB

Core philosophy

“Resilience is not about preventing failures; it is about designing systems that fail gracefully and recover quickly. If we have not tested a failure mode, we are not resilient to it — and if the test cannot fail, we have not tested it.”

1. Executive Summary
   1.1 Purpose
   1.2 The framework at a glance
   1.3 The durability principle
   1.4 How to read this document
   2. What Resilience Testing Is — Definition and Scope
   2.1 Working definition
   2.2 The five measurable properties
   2.3 Scope boundaries
3. Systems Under Test
   3.1 ShaktiDB — HA cluster (Need to update based on Recommended architecture)
   3.2 DistDB — distributed cluster (Need to update HA Architecture)
   3.3 The key structural insight
   3.5 Support contract — guarantee versus precondition
   3.6 Environment Qualification Suite (EQS)
4. The Unified Three-Tier Model
   4.1 One implementation, many targets
   4.2 What the model requires of the harness
5. Coverage Matrix
6. SLA Targets and Measurement
   6.1 SLA tiers — targets are conditional on qualified infrastructure
   6.2 Metric definitions
   6.3 RTO must be decomposed
   6.4 RPO must be measured by transaction markers
   6.5 "First write" is not recovery
7. Scenario Specification and Execution Standard
   7.1 The scenario record
   7.2 Execution lifecycle
   7.3 Environment sensitivity controls the combinatorial explosion
   7.4 Fault injection without containerization
   7.5 The workload dimension is mandatory
8. Priority Method
9. Coverage Summary by Tier
10. Tier 1 — Node-Local Scenarios (NL)
   10.1 NL-D · Durability — the fsync-truth class
   10.1a NL-E · Encryption and Key Management
   10.2 NL-C · Crash and recovery
   10.3 NL-W · WAL and archiving
   10.4 NL-I · Integrity and corruption
   10.5 NL-R · Resource exhaustion
   10.6 NL-N · Noisy neighbour and contention
   10.7 NL-M · Maintenance, vacuum and wraparound
11. Tier 2 — Cluster Scenarios (CL)
   11.1 CL-F · Failover and promotion
   11.2 CL-S · Split-brain and fencing
   11.3 CL-R · Replication resilience
   11.4 CL-N · Network
   11.5 CL-C · Connection layer
   11.6 CL-B · Backup, restore and disaster recovery
   11.7 CL-X · Compound failures
12. Tier 3 — Distributed Scenarios (DX)

12.1 DX-T · Two-phase commit and in-doubt transactions
12.2 DX-M · Metadata and coordinators
12.3 DX-S · Shard placement and rebalancing
12.4 DX-Q · Distributed query execution
12.5 DX-P · Cross-node partition
12.6 DX-D · Multi-datacentre disaster recovery
13. Design Standards
13.1 Durability cannot be validated by process kill
13.2 Acceptance criteria must be falsifiable
13.3 Failover suppression is a first-class requirement
13.4 Noisy neighbour and grey failure are a distinct class
13.5 Consistency checking must reach transaction-isolation anomalies
13.6 One machine-readable source of truth
13.7 Catalog validation — checks applied
14. Scope Boundaries and Deferred Items
16. Deliverables, Entry and Exit Criteria
16.1 Deliverables
16.2 Entry criteria
16.3 Exit criteria
17. Risk Register
18. Constraints Already Settled
19. Open Decisions Required
20. Document Approval

1. Executive Summary

1.1 Purpose

This framework establishes a single, automated approach to validating the resilience of both our *ShaktiDB standalone/HA* deployments and our DistDB distributed cluster. One catalog, one harness and one set of measurement definitions serve both systems.

It exists to prove five things, by measurement rather than assertion:

- That acknowledged commits survive real failure — including storage failure and power loss, not merely process termination
- That service is restored automatically within a defined RTO
- That data integrity is preserved, and that corruption is always detected and never silent
- That the cluster does **not** fail over when it should not
- ·That every claim is backed by audit-ready, machine-verifiable evidence (test scope and acceptance thresholds depend on deployment architecture, database size and transaction volume)

1.2 The framework at a glance

Measure	Value	Meaning
Distinct scenarios	**162**	Authored once; reused across systems and node roles
P0 critical	**104**	Must pass — blocks release
P1 high	54	Must pass or carry documented, accepted remediation
P2 medium	4	Should pass — tracked
Reuse tiers (Defined at Unified Resilience Testing Framework for ShaktiDB / DistDB | 4. The Unified Three Tier Model)	**3**	Node-local (NL) → cluster (CL) → distributed (DX)
Categories	21	Eight NL, seven CL, six DX
Systems covered	**2**	*ShaktiDB* / standalone PostgreSQL, and DistDB
Apply to standalone	**116**	Tier 1 (61) + Tier 2 (55)
Apply to DistDB	**162**	All three tiers
Target RPO — Platinum and Gold	**0**	Conditional on qualified infrastructure (§6.1)

1.3 The durability principle

Read before approving any durability claim

A `kill -9` test cannot validate durability. Killing the postmaster terminates the process but leaves the operating system page cache fully

intact — the kernel still flushes those dirty pages to disk afterwards. Such a test exercises PostgreSQL's WAL replay path and **nothing below it**.

A cluster configured with fsync = off, synchronous_commit = off, a write-back controller cache without battery backup, or a hypervisor in cache=unsafe mode passes a process-kill test **every single time**, while losing committed data in a real power event.

Category **NL-D** (§10.1, six scenarios) exists to close exactly this gap. It is the highest-value work item in the programme and is sequenced first (§15). Until NL-D passes on a given storage class, **no RPO = 0 claim may be made for that storage class**.

1.4 How to read this document

If you are…	Read
An executive or programme sponsor	§1 Executive Summary, §5 Coverage Matrix, §6.1 SLA tiers, §15 Roadmap, §17 Risk Register, §18 Open Decisions
A product owner or in sales engineering	§3.4 Environment classes, §3.5 Support contract, §3.6 Environment Qualification Suite, §6.1 SLA tiers
An engineer building the harness	§4 Three-tier model, §7 Scenario specification standard, §13 Design standards
An engineer executing tests	§10–12 the scenario catalog, §6.2–6.5 metric definitions, §16 entry and exit criteria
An auditor	§2 Definition and scope, §6 SLA measurement, §13.7 Catalog validation, §16.1 Deliverables

A note on tiers. The three reuse tiers referenced in §1.2 (Node-local (NL) → Cluster (CL) → Distributed (DX)) are defined in full in Unified Resilience Testing Framework for ShaktiDB / Dist DB | 4. The Unified Three Tier Model . Refer to Unified Resilience Testing Framework for Sha

ktiDB / DistDB | 4. The Unified Three Tier Model for the scope, applicability, and scenario counts of each tier.

2. What Resilience Testing Is — Definition and Scope

2.1 Working definition

> **Database resilience testing** is the empirical validation that a database preserves its **durability** and **integrity** contracts and restores its **availability** and **performance** contracts within stated bounds, when subjected to the failure modes to which its deployment is actually exposed.
>
> or
>
> **Database resilience testing** is the controlled validation that ShaktiDB and DistDB preserve their durability, integrity and transactional-consistency requirements, and restore availability and useful performance within approved limits, under failure conditions applicable to the deployment profile being tested.

2.2 The five measurable properties

Every scenario in this framework measures at least one of the following. A test that measures none of them is not a resilience test and does not belong in this catalog.

Property	Question it answers	Primary metric
Durability	Do acknowledged commits survive?	RPO — transactions lost
Integrity	Is stored data still correct and self-consistent?	Corruption count, invariant violations
Availability	Does service return, automatically, in time?	RTO to first successful write
Performance recovery	Does service return to *useful* levels?	RTO-to-SLO

Detectability	Was the failure noticed and signalled?	MTTD, alert precision and recall
Transactional consistency		Distributed/isolation anomalies

2.3 Scope boundaries

In scope

. Unplanned component failure

· Resource exhaustion

· Network fault

· Storage fault

· Data corruption

· Dependency failure

and the availability impact of *planned* change (rolling restart, upgrade, scaling) — because planned change is a leading source of unplanned outage.

Out of scope

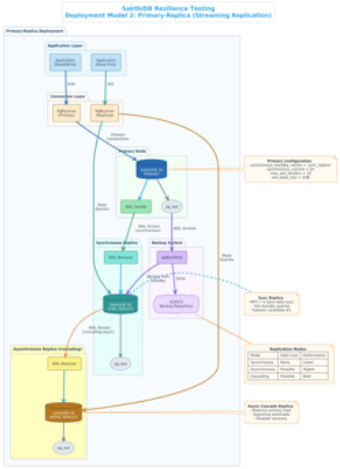
SQL and functional correctness · pure throughput benchmarking (workload is a *substrate* here, not the object of study) · security and penetration testing · capacity planning · validating monitoring configuration as an end in itself.

Detection latency is measured *inside* scenarios as MTTD; standalone "does the alert fire" tests are not resilience tests.

3. Systems Under Test

3.1 ShaktiDB — HA cluster (Need to update based on Recommended architecture)

Example Architecture



plantuml

```
!theme cerulean-outline
skinparam backgroundColor #FEFEFE
skinparam componentStyle rectangle
skinparam defaultTextAlignment center

' Global text color settings for better contrast
skinparam component {
    FontColor #2D3748
    BackgroundColor #EDF2F7
    BorderColor #4A5568
}

skinparam database {
    FontColor #FFFFFF
    BackgroundColor #2C5282
    BorderColor #1A365D
}

skinparam storage {
    FontColor #2D3748
    BackgroundColor #E2E8F0
    BorderColor #718096
}

skinparam node {
    FontColor #2D3748
    BackgroundColor #EDF2F7
    BorderColor #4A5568
}

skinparam package {
    FontColor #1A365D
    BackgroundColor #F7FAFC
    BorderColor #CBD5E0
}

skinparam note {
    FontColor #2D3748
    BackgroundColor #FFFAF0
    BorderColor #ED8936
}

skinparam cloud {
    FontColor #2D3748
    BackgroundColor #E9D8FD
    BorderColor #805AD5
}
```

```
48
49  title SakthiDB Resilience Testing\nDeployment Model 2: Primary-
    Replica (Streaming Replication)
50
51  ' ==================== PRIMARY-REPLICA DEPLOYMENT
    ====================
52  package "Primary-Replica Deployment" {
53
54      package "Application Layer" #EBF8FF {
55          component "Application\n(Read/Write)" as app_rw #BEE3F8
56          component "Application\n(Read Only)" as app_ro #BEE3F8
57      }
58
59      package "Connection Layer" #FFFAF0 {
60          component "PgBouncer\n(Primary)" as pgb_primary #FEEBC8
61          component "PgBouncer\n(Replicas)" as pgb_replica #FEEBC8
62      }
63
64      package "Primary Node" #F0FFF4 {
65          database "SakthiDB 16\nPRIMARY" as pg_primary #2B6CB0
66          component "WAL Sender" as wal_sender #9AE6B4
67          storage "pg_wal" as primary_wal #E2E8F0
68      }
69
70      package "Synchronous Replica" #E6FFFA {
71          database "SakthiDB 16\nSYNC REPLICA" as pg_sync #319795
72          component "WAL Receiver" as wal_recv_sync #81E6D9
73          storage "pg_wal" as sync_wal #E2E8F0
74      }
75
76      package "Asynchronous Replica (Cascading)" #FEFCBF {
77          database "SakthiDB 16\nASYNC REPLICA" as pg_async #B7791F
78          component "WAL Receiver" as wal_recv_async #FAF089
79          storage "pg_wal" as async_wal #E2E8F0
80      }
81
82      package "Backup System" #FAF5FF {
83          component "pgBackRest" as pgbackrest #D6BCFA
84          cloud "S3/GCS\nBackup Repository" as backup_repo
85      }
86  }
87
88  ' Application connections
89  app_rw -[#2B6CB0]-> pgb_primary : "R/W"
90  app_ro -[#319795]-> pgb_replica : "R/O"
91
92  pgb_primary -[#2B6CB0]-> pg_primary : "Primary\nConnections"
93  pgb_replica -[#319795]-> pg_sync : "Read\nQueries"
94  pgb_replica -[#B7791F]-> pg_async : "Read\nQueries"
95
96  ' Replication connections
97  pg_primary --> wal_sender
98  wal_sender -[#38A169,thickness=3]-> wal_recv_sync : "  WAL
    Stream\n  (synchronous)  "
99  wal_recv_sync --> pg_sync
100
101 pg_sync -[#DD6B20,thickness=2]-> wal_recv_async : "  WAL Stream\n
    (cascading async)  "
102 wal_recv_async --> pg_async
103
104 ' WAL storage
105 pg_primary -[#718096]-> primary_wal
106 pg_sync -[#718096]-> sync_wal
107 pg_async -[#718096]-> async_wal
108
109 ' Backup connections
110 pgbackrest -[#805AD5]-> pg_sync : "Backup from\nStandby"
```

```
pgbackrest -[#805AD5]-> backup_repo : "Store"
primary_wal -[#805AD5]-> pgbackrest : "WAL Archive"

' Notes
note right of pg_primary
  <b>Primary Configuration</b>
  - synchronous_standby_names = 'sync_replica'
  - synchronous_commit = on
  - max_wal_senders = 10
  - wal_keep_size = 1GB
end note

note right of pg_sync
  <b>Sync Replica</b>
  - RPO = 0 (zero data loss)
  - Hot standby queries
  - Failover candidate #1
end note

note right of pg_async
  <b>Async Cascade Replica</b>
  - Reduces primary load
  - Reporting workloads
  - Disaster recovery
end note

note bottom of backup_repo
  <b>Replication Modes</b>
  | Mode | Data Loss | Performance |
  | Synchronous | None | Lower |
  | Asynchronous | Possible | Higher |
  | Cascading | Possible | Best |
end note

@enduml

```

3.2 DistDB — distributed cluster (Need to update HA Architecture)

Distributed database: a Primary Coordinator holding global metadata, horizontally scaled stateless Query Coordinators, and sharded worker clusters, with an optional cross-datacenter disaster-recovery site.

The architectural properties below are confirmed against the DistDB design document and drive the whole of Tier 3.

Distributed Database HA Architecture, Installation and Operation Guide - BharatDB Project - Conflue...

Property	Behavior
Two-phase commit	**Implemented** — mandatory for DDL and metadata, optional for DML

Worker topology	4-node clusters (1 master + 3 replicas), synchronous replication, automatic promotion
Consistency model	Strong consistency via 2PC when enabled; per-shard atomicity when disabled
Transaction tracking	64-bit GXID
Primary Coordinator	Exactly one; holds global metadata, the exclusive metadata lock, and the 2PC transaction manager
Query Coordinators	Stateless; local metadata cache invalidated by the Primary Coordinator
Reference tables	Replicated to all workers via 2PC
Consensus	**No** Raft or quorum protocol for metadata — a strict single-primary policy

3.3 The key structural insight

> DistDB is ShaktiDB-derived. **Its Primary Coordinator, its Query Coordinators and every worker node are ShaktiDB instances.**
>
> Decomposing this work by product would therefore duplicate every node-level **ShaktiDB** failure mode across two catalogs, where the copies would promptly drift apart. The correct decomposition is by **scope of failure**, not by product. That is the three-tier model in section 4.

These databases ship as a **product**. Customers deploy them on infrastructure we do not own, choose or observe — dedicated and shared alike, and both must be supported. The environment is therefore a **first-class test axis**, not a fixed assumption.

> **The constraint that shapes everything below**

On infrastructure we do not own, we cannot inject a noisy neighbour — we do not control the neighbour. Category NL-N can therefore only be *executed* in our own lab, across representative environment classes. What ships to the customer is not the injection; it is **detection, graceful degradation, honest SLA tiering, and a qualification tool** they can run against their own infrastructure.

The same applies to power loss: we can force-stop our lab VMs, but we cannot cut power on a customer's managed cloud instance.

Class	Compute	Storage	Co-tenancy	Power-loss injectable?	Noisy neighbour injectable?
E1	Dedicated bare metal	Local NVMe / SSD, BBU RAID	None	Yes — physical cut plus IPMI. Strongest evidence.	External: N/A by design. **Internal still applies** — NL-N-06 (backup) and NL-N-07 (analytics) contend on any host
E2	VM on a dedicated host	Local or SAN	None (host dedicated)	Yes — hypervisor force-off	Yes — from co-located VMs we control
E3	Shared / multi-	Network block	**Yes, and invisible**	Cloud force-stop	**No** — the neighbour

	tenant cloud VM	(EBS-class)		— weaker evidence	is not ours
E4	Burstable / credit-based instance	Network block plus burst credits	Yes, plus credit cliffs	Cloud force-stop	Partially — credit depletion is self-inflictable
E5	Containerized / Kubernetes	—	—	**Not applicable — the database is not containerized. Out of scope until it is.**	—

Storage class is the independent variable for durability

Durability is determined by the *storage* stack, not the compute class;

Aailability is determined by *compute and network* predictability.

These are independent preconditions and should be qualified separately. A customer on dedicated bare metal with a lying RAID controller has worse durability than one on a shared cloud VM with a correctly-flushing network volume.

Storage classes to qualify:

. Local NVMe

· Local SSD behind BBU RAID

· SAN · network block (EBS-class)

· NFS or other network filesystem.

3.5 Support contract — guarantee versus precondition

Because the customer chooses the infrastructure, several SLA commitments are **conditional**. Stating precisely which, and shipping the tool that measures the condition, is the core product obligation this framework serves.

Property	We guarantee unconditionally	Only if customer infrastructure provides	Verified by
RPO = 0	Committed transactions survive process crash, node crash and failover, given honest storage	Storage honours fsync/FUA · no volatile write cache without BBU · fsync=on and synchronous_commit=on not overridden · sync standby healthy	NL-D-01 … NL-D-06 → **EQS-D**
RTO	Automatic detection, election and promotion within the budget for the qualified tier	CPU steal below threshold · storage p99 fsync below threshold · network RTT below threshold to peers and to the consensus store	CL-F-08 → **EQS-P**
No split-brain	Correct fencing and self-demotion in every partition scenario tested	Consensus store on an independent failure domain · clock skew within bound	CL-S-01 … CL-S-06

Integrity	Corruption is detected and surfaced, never silent	data_checksums =on at initdb time	NL-I-01 … NL-I-09
Encrypted-at-rest	Never silently unencrypted; data is never served as valid when the key is corrupted	nothing; this is unconditional	NL-E-01 … NL-E-07
Degraded-mode behaviour	**Fully ours.** Under contention we detect, declare degradation, and do not fail over spuriously — on any infrastructure	— nothing; this is unconditional	NL-N-01 … NL-N-08

The asymmetry is deliberate and worth stating to customers. We cannot promise *performance* on infrastructure we do not control — but we can and do promise *correct behaviour under degradation*: no data loss beyond the qualified RPO, no spurious failover, no silent corruption, and an honest declaration of degraded state. Category NL-N is what discharges that promise, which is why it is unconditional in the table above.

3.6 Environment Qualification Suite (EQS)

A packaged subset of this framework, runnable by a Support engineer or by the customer against *their* infrastructure before go-live. It converts “which SLA tier does this deployment qualify for?” from a sales conversation into a measurement.

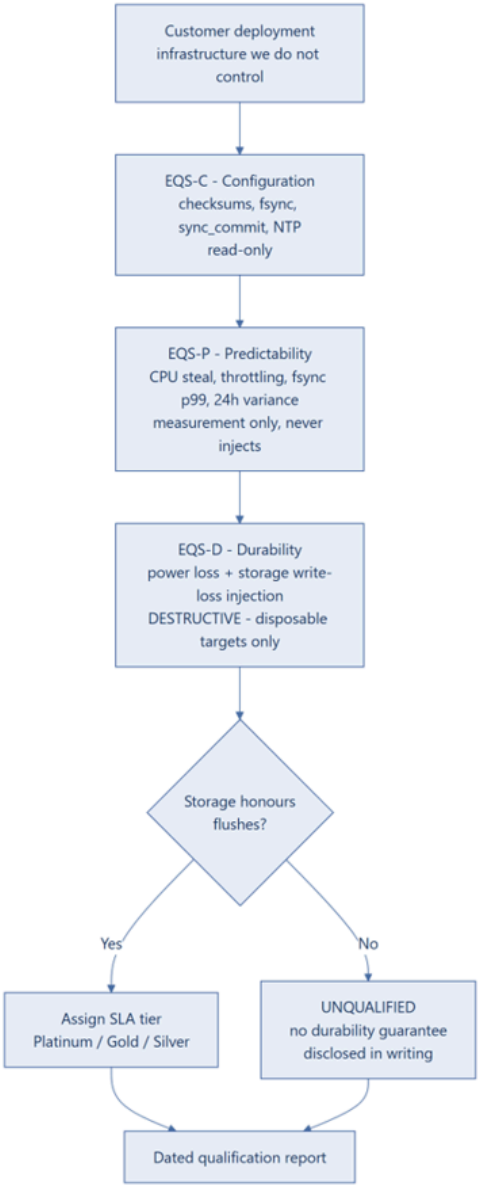


Figure 3 — Environment Qualification Suite: gating flow from deployment to assigned SLA tier.

Suite	Derived from	Measures	Gates
EQS-D — Durability	NL-D-01, 02, 03, 05, 06	fsync honesty, power-loss survival, torn-page handling, write-cache volatility	**RPO = 0 eligibility.** Fail ⇒ no durability guarantee may be offered
EQS-P — Predictability	NL-N-01, 02, 03, 08	CPU steal %, cgroup throttling,	**RTO tier assignment**

	(measurement-only variants)	fsync p99, sustained variance over 24 h	
EQS-C — Configuration	NL-I-09 plus configuration assertions	checksums, fsync, synchronous_commit, NTP, consensus-store placement	Any guarantee at all

Output

A dated qualification report naming the SLA tier the environment supports, the measured values behind it, and any disclosed limitations. It is re-run on any infrastructure change — instance family, storage class, hypervisor, or host move. This is a *product deliverable*, not an internal test artefact.

Note that EQS-P is **measurement-only**: it observes the steal, throttling and variance that the customer's real neighbours produce. It does not inject anything, because on shared infrastructure we cannot.

4. The Unified Three-Tier Model

Scenarios are organised by **scope of failure**: what breaks, not which product it broke in. A node-local fault is a node-local fault whether the node is a standalone primary or a DistDB worker replica.

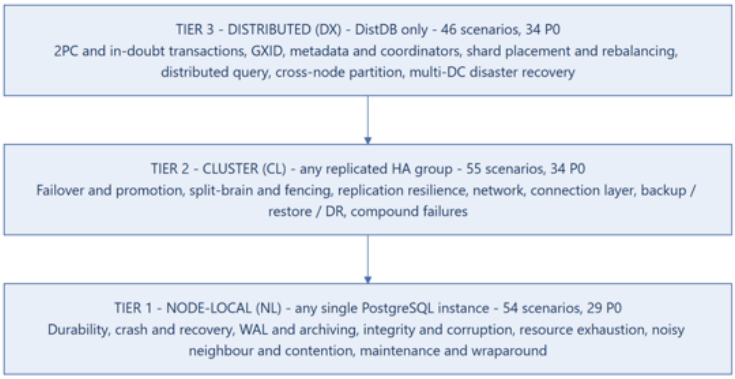


Figure 4 — The three tiers. Each tier adds scope; each inherits everything below it.

- **Tier 1 — NL, node-local.** Applies to any single ShaktiDB instance. 61 scenarios, 36 of them P0.
- **Tier 2 — CL, cluster.** Applies to any replicated HA group. 55 scenarios, 34 of them P0.
- **Tier 3 — DX, distributed.** DistDB only. 46 scenarios, 34 of them P0.

4.1 One implementation, many targets

A Tier-1 scenario is authored once. The harness's target resolver then executes it against the standalone instance, against the DistDB Primary Coordinator, against each Query Coordinator, and against every worker node. The *test code* does not change — only the target selector.

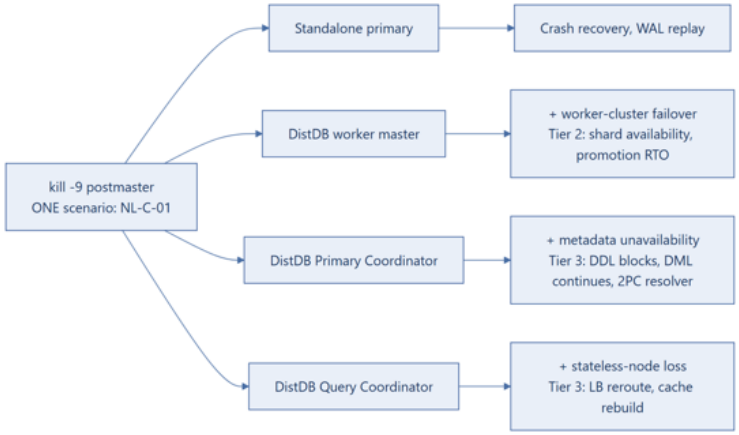


Figure 5 — Topological amplification: one authored scenario, four targets, four assertion sets.

kill -9 postmaster is **one** scenario, NL-C-01:

Applied to	Exercises	Additional assertion layered on
Standalone primary	Crash recovery, WAL replay	—
DistDB worker master	Crash recovery *and* worker-cluster failover	Tier 2: shard availability, promotion RTO
DistDB Primary Coordinator	Crash recovery *and* metadata unavailability	Tier 3: DDL blocks, DML continues, 2PC resolver
DistDB Query Coordinator	Crash recovery *and* stateless-node loss	Tier 3: load-balancer reroute, cache rebuild

Same fault, same code, four different assertion sets — rather than four separately authored scenarios that drift apart over time.

4.2 What the model requires of the harness

This model imposes exactly one requirement on the harness: **a target resolver that can enumerate PostgreSQL/Shakti instances by role.** Given applies_to: [distdb.worker], it must expand to every worker master and replica and run the scenario against each. Resolver strategies are needed for Patroni, for label selectors, for direct host lists, and for DistDB node roles.

5. Coverage Matrix

System	Tier 1 (NL)	Tier 2 (CL)	Tier 3 (DX)	Distinct scenarios	Execution instances
Standalone PostgreSQL / ShaktiDB	✔ 61	✔ 55 (if HA)	—	**116**	~116
DistDB	✔ 61 × 4 roles	✔ 55 × N clusters	✔ 46	**162**	~323+

162 scenarios cover both systems. Because the decomposition is by scope of failure rather than by product, there is no cross-catalog duplication to maintain, and every node-level failure mode is exercised against every node role.

"Execution instances" counts *runs*, not scenarios to author and maintain.

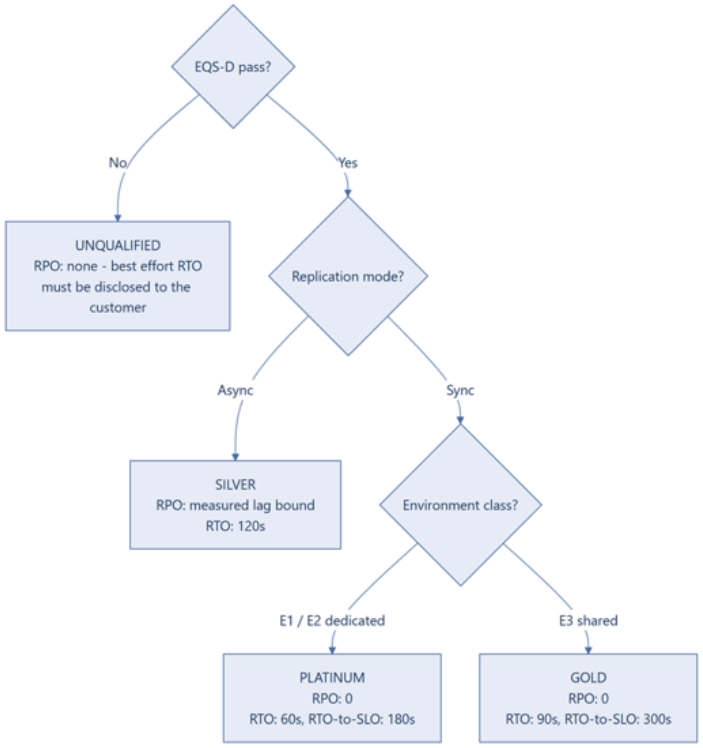
Tier	Categories	Scenarios	P0	P1	P2
TIER 1 — NL	Durability ·Encryption Crash · WAL ·	**61**	36	23	2

	Integrity · Resources · Noisy Neighbour · Maintenance				
TIER 2 — CL	Failover · Split-brain · Replication · Network · Connection · Backup/DR · Compound	**55**	34	19	2
TIER 3 — DX	2PC · Metadata · Shards · Query · Partition · Multi-DC	**46**	34	12	0
TOTAL	**21 categories**	**162**	**104**	**54**	**4**

6. SLA Targets and Measurement

6.1 SLA tiers — targets are conditional on qualified infrastructure

A single unconditional SLA is not honest for a product deployed on customer-chosen infrastructure. Targets are therefore **tiered by qualified environment class**, with the tier assigned by the Environment Qualification Suite (§3.6) rather than asserted in a contract.



Tier	Qualifying environment	RPO	RTO	RTO-to-SLO	Notes
Platinum	E1 / E2 · EQS-D pass · sync replication	0	≤ 60 s	≤ 180 s	Full guarantee. Dedicated infrastructure, honest storage.
Gold	E3 general purpose · EQS-D pass · sync replication	0	≤ 90 s	≤ 300 s	RPO fully preserved; RTO relaxed to absorb shared-compute variance.
Silver	E3 / E4 · EQS-D	≤ measured	≤ 120 s	Best effort	RPO > 0 *by design*

	pass · async replication	lag bound			*choice*, not by infrastructure defect. The bound must be measured and disclosed.
Unqualified	**EQS-D fail** — storage does not honour flushes	**None**	Best effort	—	**No durability guarantee may be offered. The failure and its cause must be disclosed to the customer in writing.**

The **Unqualified** row is the one that matters commercially. If a customer deploys on storage that discards flushes, no amount of correct database behaviour can deliver RPO = 0 — and a framework that reports "pass" in that situation is actively harmful. EQS-D exists to make that condition visible *before* go-live rather than after the first power event.

6.2 Metric definitions

Metric	Standalone / ShaktiDB	DistDB	Definition

RPO	Per tier	Per tier	Committed transactions lost, measured by transaction markers
RTO (first write)	Per tier	Per tier, per shard	Fault injection → first successful write
RTO-to-SLO	Per tier	Per tier	Sustained ≥ 80% baseline TPS at p99 ≤ 1.5× baseline, held 60 s
Failover time	≤ 30 s (Platinum)	≤ 30 s worker / ≤ 60 s PC	Leader change complete and accepting writes
MTTD	≤ 10 s	≤ 10 s	Fault → detection signalled. Unconditional — detection is ours.
Data integrity	100%	100%	Zero corruption, zero invariant violations. Unconditional given checksums enabled.
Spurious failover rate	0	0	Unconditional — must hold on every environment class, including E3 and E4

DR RTO / RPO	Per DR policy	Per DR policy	Measured at production data volume, never assumed

6.3 RTO must be decomposed

Measuring RTO as a single scalar hides where the budget is spent and makes tuning blind. When RTO regresses from 22 s to 48 s, one number tells you nothing.

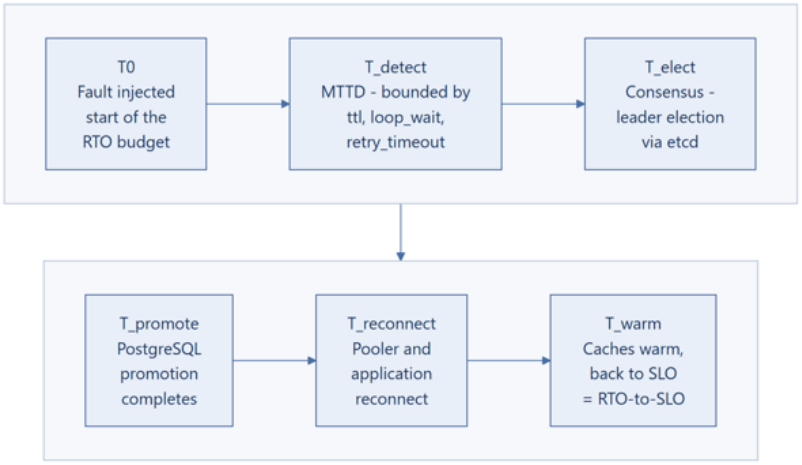


Figure 7 — The five components of RTO. Each is measured and budgeted separately.

```
RTO  =  T_detect  +  T_elect  +  T_promote  +  T_reconnect  +  T_warm
        (MTTD)      (consensus)  (PG promote)  (pooler/app)    (to SLO)
```

For a Patroni cluster, T_detect is bounded by ttl, loop_wait and retry_timeout.

This is usually where most of the RTO budget actually goes -- and it is tunable.

Every Tier-2 failover scenario records all five components separately, and each component has its own budget.

6.4 RPO must be measured by transaction markers

This is the only reliable method. Before each commit the client appends a marker to a **local journal file** and flushes it; that file — not the database — is the source of truth for what the

client believed was committed. After recovery, the two sets are differenced.

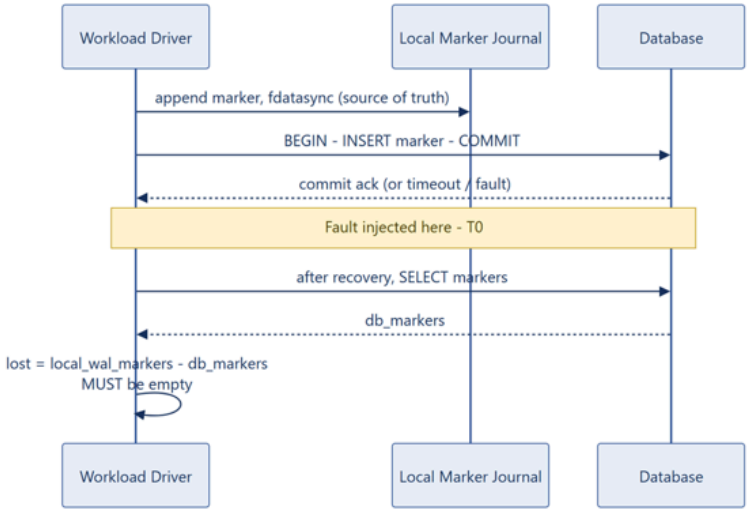


Figure 8 — Transaction-marker protocol for measuring RPO.

```
lost_transactions = local_wal_markers - database_markers    # MUST be empty
```

Any scenario measuring RPO must declare transaction_markers: true in its workload block.

6.5 “First write” is not recovery

> A freshly promoted replica has a cold shared buffer cache and a cold OS page cache. It will accept a single write within seconds while serving production traffic at a fraction of baseline throughput for several minutes. Reporting RTO-to-first-write alone materially overstates recovery. **Both figures are reported for every failover scenario.**

7. Scenario Specification and Execution Standard

Every scenario is specified with all of the fields below. A scenario missing any field is not ready to execute. This record is the machine-readable source of truth from which the catalog documents are generated.

7.1 The scenario record

```
id: NL-D-02

name: Durability under host power loss

tier: node_local
```

```
applies_to: [standalone, distdb.pc, distdb.qc, distdb.worker]

steady_state:                  # measured BEFORE injection, must hold to proceed
  tps_min: 800
  p99_latency_ms_max: 50
  replication_lag_s_max: 1
  duration_s: 120

workload:
  profile: oltp_write_heavy      # idle | oltp_read | oltp_write_heavy |
  concurrency: 64                #   bulk_load | long_analytic | mixed
  transaction_markers: true      # required for any RPO-measuring scenario

fault:
  type: host_power_loss
  method: virsh destroy          # NOT kill -9 -- see section 13.1
  target: primary
  duration: permanent

blast_radius:
  max_nodes_affected: 1
  environment: [lab, staging]    # never production
abort_if:
  - replication_lag_s > 300
  - secondary_node_unhealthy

# ---- COMPOUND SCENARIOS ONLY (category CL-X) ------------------------------
```

```
# A second fault, armed on an OBSERVED STATE rather than a timer. "20
seconds
# in" and "the instant promotion begins" are different tests; only the
second
# reliably hits the fragile window.
compound:
  fault_b:
    trigger: on_state           # on_state | on_metric | after_phase
    condition: patroni_state == "promoting"
    timeout_s: 120              # if the state never arrives, abort -- do not
                                #   silently degrade to a single-fault test
    type: process_kill
    target: sync_standby
  blast_radius_override:
    max_nodes_affected: 2       # documented exception; 1 everywhere else
# ------------------------------------------------------------------------
--

measure: [rpo_txn, rto_first_write_s, rto_to_slo_s, mttd_s,
          corruption_count, t_detect_s, t_elect_s, t_promote_s]

accept:                         # every predicate numeric or boolean
  - rpo_txn == 0
  - rto_first_write_s <= 60
  - rto_to_slo_s <= 180
  - corruption_count == 0
  - amcheck_errors == 0

environment:                    # the deployment axis
```

```
  classes: [E1, E2]          # where this scenario can actually be executed
  env_sensitive: true        # does the RESULT depend on environment class?
                             #   true  → must run on every supported class
                             #   false → run once on the reference environment

priority: P0
```

7.2 Execution lifecycle

Every run follows the same eight-phase machine. Phases are bounded by timeouts; an unbounded wait is a defect.

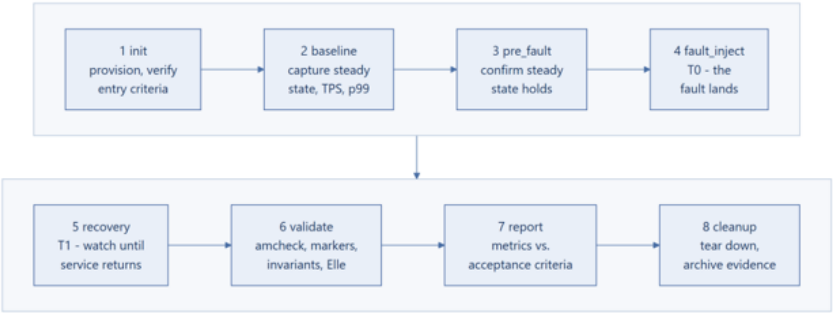


Figure 9 — The eight-phase test lifecycle. T0 is fault injection; T1 is the start of recovery measurement.

> **Safety is enforced by the harness, not by convention**
>
> Every scenario declares a blast radius and abort conditions. The Safety Controller enforces a kill switch, a circuit breaker, a maximum-affected-nodes cap, and a timeout on every operation. The cap is **one node** for every scenario except the compound category (§11.7), where it is two.

7.3 Environment sensitivity controls the combinatorial explosion

162 scenarios × 4 environment classes would be 648 runs, which is unmanageable and mostly redundant. Most scenarios are **environment-insensitive**: a CLOG corruption test behaves identically on bare metal and on a shared cloud VM, so it runs once on the reference environment.

Only **environment-sensitive** scenarios multiply — those whose *result*, not merely their execution, depends on the infrastructure underneath:

Category	Sensitive?	
NL-D Durability	**Yes — all**	The entire point is measuring the storage stack. Must run on every storage class offered.
NL-N Noisy neighbour	**Yes — all**	Contention behaviour is the property under test.
CL-F Failover	Partly	The RTO budget shifts with compute and network variance. CL-F-01/02/03/08 are sensitive; the rest are not.
CL-B Backup and restore	Partly	CL-B-05 (restore RTO) is throughput-bound and therefore sensitive.
NL-C, NL-W, NL-I, NL-M, NL-E	No	PostgreSQL-internal behaviour. Reference environment only.
CL-S, CL-R, CL-C, CL-N	No	Protocol and configuration behaviour, not infrastructure behaviour.
DX-∗ (all)	No, except DX-S-08 and DX-Q-07	Distributed protocol behaviour is infrastructure-independent; the two co-tenancy scenarios are not.

Result

21 environment-sensitive scenarios (NL-D 6 + NL-N 8 + CL-F-01/02/03/08 + CL-B-05 + DX-S-08 + DX-Q-07) × 4 classes = 84 runs, plus the remaining 141 run once = 225 runs rather than 648 — and the multiplied set is exactly the set a customer cares about when asking "will this work on *my* infrastructure?"

7.4 Fault injection without containerization

The database is not containerized

Chaos Mesh and LitmusChaos are Kubernetes-native: they inject faults via custom resources against pods. Neither is usable against our deployment model today, and neither can be the primary injection path.

One important qualification: the Chaos Mesh project also ships chaosd, a standalone agent and CLI for **physical machines and VMs** — process kill, network, disk, stress, clock and JVM faults, with no Kubernetes involved. chaosd is directly applicable to E1–E4 and is a credible alternative to hand-rolling the injection layer. LitmusChaos has no equivalent and is genuinely Kubernetes-only.

Tool	Usable today?	Notes
Chaos Mesh (Kubernetes controller and CRDs)	No	Requires Kubernetes; a CRD-builder adapter is deferred until E5 exists
`chaosd` (Chaos Mesh, standalone)	**Yes**	Physical and VM agent. Covers most of the table below behind one interface — evaluate before building bespoke adapters

LitmusChaos	No	Kubernetes-only, with no standalone mode. Deferred indefinitely.
Direct OS tooling (SSH, systemd, tc, dm-*, libvirt)	Yes	Always available; the fallback and the reference implementation

Fault class	Non-containerized — use today	Containerized — future, if and when E5 arrives
Process kill	SSH plus kill -9 / systemctl kill -s SIGKILL	Chaos Mesh PodChaos
CPU / memory limits	systemctl set-property CPUQuota=, MemoryMax=, IOReadBandwidthMax=	Chaos Mesh StressChaos, Kubernetes limits
Network	tc netem, iptables / nftables	Chaos Mesh NetworkChaos
Storage / I/O	dm-flakey, dm-delay, dm-error, fio contention, blkio cgroup	Chaos Mesh IOChaos
Node / power	virsh destroy, IPMI power-off, cloud force-stop API	Node drain or eviction
Clock	chronyc, date, libfaketime	Chaos Mesh TimeChaos

cgroups do not require containers

systemd places every service in a cgroup by default on any modern Linux distribution, so systemctl set-property postgresql.service CPUQuota=50% reproduces CFS throttling on a plain VM. NL-N-02 is therefore fully testable today — throttling simply becomes *more* likely later, if the product is containerized.

Recommendation: build the SSH/systemd/tc/dm-* injection path as primary, behind a FaultInjector adapter interface, so that container-native injectors can plug in unchanged if containerization lands later.

7.5 The workload dimension is mandatory

The same fault produces entirely different outcomes against an idle database, a 5,000 TPS OLTP workload, a long-running analytic query, and a bulk COPY. Workload is therefore a declared parameter of every scenario — never chosen ad hoc at execution time.

8. Priority Method

A four-factor weighted score, applied uniformly to both systems and to all three tiers.

Factor	Weight	High (3)	Medium (2)	Low (1)
Business Impact	40%	Data loss, complete outage, SLA breach	Partial outage, degraded performance	Minor disruption, no data impact
Likelihood	25%	Common failure mode (hardware, network)	Occasional occurrence	Rare edge case
Detection Difficulty	20%	Silent failure, delayed detection	Requires cross-system correlation	Immediate and obvious
Recovery Complexity	15%	Manual intervention,	Automated but complex	Simple automatic

		data recovery		recovery

Priority	Score	Count	Gate
P0 Critical	2.5 – 3.0	104	Must pass — blocks release
P1 High	1.8 – 2.4	54	Must pass, or carry documented remediation
P2 Medium	1.0 – 1.7	4	Should pass — tracked

9. Coverage Summary by Tier

Section 5 gave the totals. This section summarizes coverage one level down — category by category, with a one-line description of what each actually tests — so the detail in §10–12 can be read against a map rather than discovered cold.

Category	What it tests	Scenarios	P0
TIER 1 — NODE-LOCAL · 61 scenarios, 36 P0 · any single PostgreSQL instance			
Durability	Storage and power-loss durability — the fsync-truth class	6	**5**
Encryption & Key Management	KMS/keystore unavailability, key corruption, interrupted	7	7

	rotation, encrypted page/WAL recovery, key-access permission loss		
Crash & Recovery	Process termination, checkpoint, and large-transaction crashes	7	**4**
WAL & Archiving	WAL segment integrity and archive continuity	7	**4**
Integrity & Corruption	Detection of heap, index, CLOG and control-file corruption	9	**6**
Resource Exhaustion	Memory, disk, connection and file-descriptor limits	9	**4**
Noisy Neighbour	CPU steal, cgroup throttling, burst-credit depletion, grey failure	8	**4**
Maintenance & Wraparound	Vacuum, autovacuum starvation, XID and multixact wraparound	8	**2**

TIER 2 — CLUSTER · 55 scenarios, 34 P0 · any replicated HA group			
Failover & Promotion	Automatic recovery within RTO, including failover suppression	10	**7**
Split-Brain & Fencing	No two nodes ever accept writes at once	6	**4**
Replication	Sync and async replication, quorum degradation, slot bloat	10	**6**
Network	Latency, packet loss and partition at the cluster level	6	**1**
Connection Layer	Pooler behaviour and reconnect storms during failover	5	**2**
Backup, Restore & DR	Backup integrity, PITR accuracy, restore at production volume	9	**5**
Compound Failures	Two concurrent faults — the	9	**9**

	second injected when the system is most fragile		
TIER 3 — DISTRIBUTED · 46 scenarios, 34 P0 · DistDB only			
Two-Phase Commit	In-doubt transactions, coordinator failure mid-commit	11	**8**
Metadata & Coordinators	Primary and Query Coordinator failure, metadata consistency	9	**7**
Shard Placement	Rebalancing, node loss, reference-table replication	8	**6**
Distributed Query	Partial results, stragglers, repartition failures	7	**4**
Cross-Node Partition	Network splits inside the cluster, convergence on healing	6	**6**

Multi-DC Disaster Recovery	Cross-datacentre failover, DR RTO and RPO	5	**3**
TOTAL — 21 categories		**162**	**104**

Standalone PostgreSQL / ShaktiDB runs Tiers 1 and 2 (116 scenarios). DistDB runs all three tiers (162), with every Tier-1 category above re-executed against the Primary Coordinator, each Query Coordinator, and every worker role (§4.1).

10. Tier 1 — Node-Local Scenarios (NL)

pplies to every single PostgreSQL instance — standalone ShaktiDB , the DistDB Primary Coordinator, every DistDB Query Coordinator, and every DistDB worker master and replica. Authored once, executed against all.

10.1 NL-D · Durability — the fsync-truth class

> **No RPO = 0 claim is valid until this category passes.** These are the only tests in the framework that can detect a dishonest storage stack.

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri
NL-D-01	fsync honesty baseline *(screening diagnostic)*	pg_test_fsync plus diskchecker against the data and WAL volumes	Automatic	Screening diagnostic — no recovery needed. pg_test_fsync completes; record baseline. If anomalous rates detected: flag	Baseline recorded per volume and carried into the NL-D-02/03 report; anomalous flush rates flagged as *suspected* volatile cache. **Cannot by itself prove or disprove honesty** — a non-volatile cache legitimately acknowledges fast.	**P1**

				for NL-D-02/03 gating.	Gating is by NL-D-02/03.	
NL-D-02	Host power loss under write load	VM force-off / virsh destroy at 1000 TPS	Automatic	1. VM force-off (virsh destroy) terminates all processes and flushes. 2. Restart VM. Database starts automatically (systemd). 3. WAL replay from last checkpoint. Uncommitted work rolled back. 4. Database opens for connections. TESTER: Do NOT manually restart PG. If systemd auto-restart is not configured,	rpo_txn == 0; corruption_count == 0; the database starts unattended	**P0**

				reclassify as SA and note in results.		
NL-D-03	Storage write-loss injection	dm-flakey drops writes for 5 s during a commit burst	Automatic	1. dm-flakey drops writes for 5s during commit burst. 2. Database detects I/O errors during the drop window. 3. On dm-flakey restore: if PG crashed, WAL replay recovers. 4. If PG detected write loss: it PANICs and restarts. CRITICAL: If rpo_txn > 0 with no PANIC or error: the storage lied, and this is the finding (not a	rpo_txn == 0 OR startup refuses with an explicit error — never silent loss	**P0**

				recovery failure).		
NL-D-04	Durability configuration matrix	synchronous_commit ∈ {off, local, remote_write, on, remote_apply} × power loss. *off and local apply to any instance; the remaining three require a sync standby and are evaluated on clustered targets — see CL-R-04.*	Semi automatic	1. Power loss at each synchronous_commit level. 2. For off/local: measured RPO matches the 3×wal_writer_delay bound. 3. For on/remote_apply: requires sync standby for evaluation. Recovery: restart VM; WAL replay; verify RPO by marker diff.	Measured RPO matches the documented contract for each applicable level, exactly. For off, loss is **bounded by 3 × `wal_writer_delay`**, not merely “some loss”	**P0**
NL-D-05	Torn page / partial write	Power loss mid-checkpoint with full_page_writes on, then off	Manual Recovery	1. Power loss mid-checkpoint with FPW on: WAL replay repairs torn	FPW on: corruption_count == 0. FPW off: the torn page is *detected* and not silent — note this detection depends on	**P0**

				pages. Recovery type = A. 2. Power loss mid-checkpoint with FPW off: torn page may exist. 3. If data_checksums=on: torn page detected on read. Repair requires: a) Restore page from backup (page-level), or b) Full PITR restore if page cannot be isolated. 4. If data_checksums=off: torn page is SILENT. This is the finding.	data_checksums being enabled; without checksums a torn page is silent, which is itself the finding	

NL-D-06	Volatile controller cache detection	Power loss with write-back cache enabled and no BBU	Manual Recovery	1. Power loss with volatile write-back cache (no BBU). 2. Data loss likely. This is a DETECTION test, not a survival test. 3. If loss detected: document. The finding is the detection itself. 4. If loss NOT detected: P0 defect — the product failed to identify a dishonest storage stack. Recovery of the test instance: restore from pre-test backup.	The test **detects and reports** the durability violation	**P0**

10.1a NL-E · Encryption and Key Management

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri
NL-E-01	KMS/keystore unavailable at startup	Block network path to external KMS (or remove local keystore file) before `pg_ctl start`	Manual Operation	Startup blocked — DB must refuse to start with an explicit key-unavailable error. **Recovery:** restore KMS reachability / keystore access, then restart normally.	Startup fails loudly and explicitly; never falls back to starting unencrypted or with a cached/stale key; error identifies the KMS/keystore as the cause	**P0**
NL-E-02	KMS unavailable mid-session (key needed for a new segment/rotation)	Block KMS connectivity while the instance is already running and under write load	Automatic / Semi Automatic	Existing sessions using already-fetched keys continue. New operations requiring a fresh key call (e.g., segment key wrap) either block explicitly or fail explicitly.	No silent unencrypted write occurs under any circumstance; failure mode (block vs. explicit error) matches the documented contract exactly; `rpo_txn == 0` for anything	**P0**

				Recovery: restore KMS reachability; blocked operations resume.	acknowledged before the KMS loss	
NL-E-03	Corrupted or truncated key material	Flip bytes in the local keystore file, or return malformed key bytes from a mocked KMS response	Manual Recovery	Detected key-integrity check fails on load. **Recovery:** restore keystore from backup, or re-provision key from KMS source of truth (requires documented runbook — no operator improvisation).	Detected before any decrypt/encrypt attempt is made with the corrupt key; explicit error identifying key corruption; never silently decrypts into garbage and serves it as valid data (TDE analogue of NL-D-06 — a detection test, not a survival test)	**P0**
NL-E-04	Crash mid-encrypted-page-write	`kill -9` the backend/checkpointer during a write to an encrypted data page, timed via the same state-trigger mechanism as compound	Automatic	WAL replay on restart must correctly decrypt/re-apply without leaving a page in a half-encrypted, unreadable state.	`corruption_count == 0`; `amcheck_errors == 0`; page is fully readable post-recovery under the current key — no partial-cipher artifact; if `data_checks`	**P0**

		scenarios (§7.1 `compound.fault_b.trigger: on_state`)			`ums=on`, any anomaly is caught by NL-I-01's existing checksum path, not silently passed through	
NL-E-05	Key rotation interrupted mid-rotation	Kill the instance (or the rotation utility process) partway through a key-rotation operation that re-wraps existing data/segment keys	Manual Operation	On restart, the instance must detect the incomplete rotation and either resume it deterministically or refuse to start until resolved — never leave data readable under only some of the intended keys with no record of which. **Recovery:** documented rotation-resume or rollback runbook.	Rotation state is explicit and queryable (e.g., which objects are old-key vs. new-key); no object is left ambiguously keyed; no data becomes permanently unreadable as a side effect	**P0**
NL-E-06	Encrypted WAL segment corruption	Flip bytes in an unreplayed encrypted	Automatic	Restart triggers replay;	Truncation LSN reported; decryption	**P0**

		WAL segment, then restart (TDE-specific variant of NL-W-03)		corruption is detected mid-replay — decryption failure must not be silently swallowed or misread as “end of valid WAL.” Recovery ends at the corruption point with an explicit warning.	failure is distinguishable in logs from ordinary end-of-WAL; transactions lost after the corruption point are quantified, never silently assumed absent	
NL-E-07	Key-access permission failure (file permissions / IAM revoked)	Revoke filesystem read permission on the local keystore, or revoke the IAM role/credential used to reach KMS, while the instance is running	Semi Automatic	Existing sessions continue on already-cached keys where architecturally correct; new key fetches fail explicitly. **Recovery:** restore permissions/IAM grant; verify next key fetch succeeds.	Failure is attributed correctly to a permissions/IAM cause in logs (not misreported as generic KMS-down or generic I/O error); no silent degradation to an unencrypted path	**P0**

10.2 NL-C · Crash and recovery

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri
NL-C-01	Immediate process termination	kill -9 postmaster under load	Automatic	1. kill -9 postmaster terminates all backends. 2. systemd Restart=always triggers restart. 3. Startup process detects unclean shutdown; enters recovery. 4. WAL replay from last checkpoint. Uncommitted txns rolled back. 5. Database opens for connections. TESTER: Observe; do not intervene. Record rto_first_write_s.	rpo_txn == 0; rto_first_write_s ≤ 60; amcheck_errors == 0	**P0**

NL-C-02	Crash during checkpoint	kill -9 while the checkpointer is active	Automatic	1. kill -9 during active checkpoint. 2. Recovery replays WAL from PREVIOUS completed checkpoint. 3. Partial checkpoint ignored — WAL is source of truth. 4. New checkpoint issued post-recovery.	corruption_count == 0; recovery completes unattended	**P0**
NL-C-03	Crash during a large transaction	kill -9 during a 10M-row INSERT	Automatic	1. kill -9 during 10M-row INSERT. 2. WAL replay: uncommitted INSERT rolled back (MVCC). 3. Table accessible with only previously	Transaction fully absent; no partial rows; FK invariants hold; **Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies** in the recorded history at the declared isolation level	**P0**

				committed data. 4. Elle history checked for isolation anomalies.		
NL-C-04	Recovery time versus WAL volume	Crash at 1 / 4 / 16 GB of un-checkpointed WAL	Semi Automatic	1. Crash at varied WAL volumes (1/4/16 GB un-checkpointed). 2. Each restart triggers WAL replay of that volume. 3. Recovery time is proportional to WAL volume. TESTER: For 16 GB case, restart may need manual trigger if systemd timeout exceeded. Record recovery_s at each volume.	recovery_s ≤ the RTO budget at the maximum WAL volume permitted by max_wal_size; the relationship is linear and predictable, so the worst case is computable from configuration	**P0**

NL-C-05	Repeated crash cycles	10 × (kill -9, restart) at 5 s intervals	Automatic	1. 10× rapid crash-restart cycles. 2. Each cycle: WAL replay → open → kill → replay. 3. Recovery time should remain stable across cycles. TESTER: Record recovery_s per cycle. Flag if cycle 10 > 1.5× cycle 1.	recovery_s at cycle 10 ≤ 1.5 × cycle 1; no cumulative bloat	**P1**
NL-C-06	Crash during CREATE INDEX CONCURRENTLY	kill -9 mid-build	Automatic	1. kill -9 during CREATE INDEX CONCURRENTLY. 2. On recovery: index marked INVALID (pg_index.i	Index left INVALID, table readable, rebuild succeeds	**P1**

				ndisvalid = false). 3. Table remains readable; queries work without the index. POST-TEST CLEANUP: DROP invalid index; re-run CREATE INDEX.		
NL-C-07	Crash while acting as a backup source	Kill the primary while pg_basebackup is streaming	Automatic	1. kill -9 primary while pg_basebackup streams. 2. Primary recovers normally via WAL replay. 3. The backup is left incomplete on the backup client. BACKUP SIDE: The backup must	Backup fails cleanly and is detectably incomplete — never silently truncated	**P1**

				be detectably incomplete, never silently truncated.		

10.3 NL-W · WAL and archiving

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri
NL-W-01	Commit durability with markers	Marker-instrumented load, then power loss	Automatic	1. Marker-instrumented load, then power loss. 2. WAL replay recovers all flushed WAL. 3. Transaction marker diff: local_markers − db_markers.	local_markers − db_markers == 0	**P0**
NL-W-02	WAL segment boundary crash	Power loss during a segment switch	Automatic	1. Power loss during WAL segment switch.	No WAL corruption; recovery completes	**P0**

				2. Recovery handles incomplete segment boundary. 3. New segment allocated on resume.		
NL-W-03	WAL segment corruption in pg_wal	Flip bytes in an unreplayed segment, then restart	Automatic	1. Flip bytes in unreplayed WAL segment. 2. Restart triggers replay; corruption detected mid-replay. 3. Recovery ends at corruption point with explicit warning. 4. Transactions after corruption point	Truncation LSN reported and a *recovery-ended-early* warning raised; transactions lost after the corruption point quantified. **Never a silent early end to replay.**	**P0**

				are lost (quantified). POST-RECOVERY: Restore from backup for data beyond corruption point, or accept the truncated recovery.		
NL-W-04	Archive command failure	archive_command returns non-zero for 30 minutes	Manual operation	1. archive_command failing for 30 minutes. 2. WAL accumulates in pg_wal but is not archived. 3. RECOVERY: Fix archive destination (free space, fix permissions, restore network	Zero WAL loss; pg_wal stays ≤ 2 × max_wal_size plus archive backlog and never exceeds 90% of filesystem capacity; MTTD ≤ 300 s	**P1**

				to archive host). 4. archive_command resumes; backlog drains. 5. Verify no WAL gap in archive sequence.		
NL-W-05	Archive under sustained pressure	Throttle the archive to 50% of the WAL generation rate	Manual Operation	1. Archive throttled to 50% of WAL rate. 2. Backlog grows; backpressure must engage before disk fills. 3. RECOVERY: Remove throttle; archive catches up. 4. Verify no gap in archive	No gap in the archive sequence; backpressure applied before the disk fills	**P1**

				sequence.		
NL-W-06	Archive gap mid-chain	Delete a segment from the archive, then attempt PITR	Automatic	1. Delete a segment from archive, then attempt PITR. 2. Restore fails at exact gap with correct LSN reported. 3. This is a detection test — the “recovery ” is the failure message.	Restore fails at the exact gap with the correct LSN reported	**P1**
NL-W-07	pg_wal filesystem full	Fill pg_wal to 100% under load	Semi Automatic	1. pg_wal filesystem at 100% under load. 2. Database PANICs (cannot write WAL).	PANIC is clean and recoverable; corruption_count == 0; restarts unattended once space is freed	**P0**

				3. RECOVERY: Free space in pg_wal (remove old segments via pg_archivecleanup, or expand volume). 4. Restart database. WAL replay. Service resumes.		

10.4 NL-I · Integrity and corruption

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri
NL-I-01	Heap page checksum	Corrupt a byte in a heap file	Manual Recovery	1. Heap page corruption detected on read via checksum. 2. Affected rows/pages inaccessible. RECOVERY OPTIONS: a) PITR from backup to before	Detected on read; the error identifies the relation and block	**P0**

				corruption (full MR). b) SET zero_damaged_pages=on to skip corrupt pages (DATA LOSS for those pages — last resort). c) If standby exists: failover and rebuild (CL-level recovery).		
NL-I-02	Index corruption	Corrupt a btree file	Manual Operation	1. B-tree index corruption detected by amcheck. 2. Index returns wrong results or errors. RECOVERY: REINDEX restores the index from heap data. No data loss — indexes are derived from heap.	amcheck detects it; REINDEX restores; no wrong results returned first	**P0**

NL-I-03	CLOG / pg_xact corruption	Corrupt a CLOG page	Manual Operation	1. CLOG (pg_xact) page corrupted. 2. Transaction visibility broken for affected range. RECOVERY: Documented runbook using pg_resetwal or restore. pg_resetwal resets transaction status — risks data inconsistency. Preferred: PITR from backup (escalate to MR).	Detected, not silently misinterpreted; the documented recovery path succeeds	**P0**
NL-I-04	pg_control corruption or loss	Corrupt or delete pg_control	Manual Recovery	1. pg_control corrupted or deleted. 2. Database cannot start — no control file. RECOVERY RUNBOOK: a) pg_resetwal -f $PGDATA to recreate pg_control (risk: data	Recovery completes via the documented runbook with no operator improvisation; lost transactions **counted and reported**, not estimated	**P0**

				inconsistency for in-flight txns). b) Preferred: restore from backup (full MR). Lost transactions counted and reported, not estimated.		
NL-I-05	Visibility map / FSM corruption	Corrupt the _vm and _fsm forks *separately — their severities are opposite*	Semi Automatic	1. VM/FSM fork corruption. FSM: Self-heals (rebuilt by vacuum). Recovery = A. VM: Falsely-set all-visible bit — detected by pg_check_visible(). RECOVERY: VACUUM the affected table to rebuild VM.	**FSM:** self-heals (not WAL-logged, rebuilt); no wrong results. **VM:** a falsely-set all-visible bit **must not cause index-only scans to return dead rows**; detected by pg_visibility's pg_check_visible() / pg_check_frozen() — *not* by amcheck, which does not verify the visibility map — and vacuum must not skip the page	**P1**
NL-I-06	TOAST corruption	Corrupt a TOAST chunk	Manual Recovery	1. TOAST chunk corruption. Detected on detoast.	Detected on detoast; the error identifies the row	**P1**

				2. Affected row's large value inaccessible. RECOVERY: Restore affected table from backup, or accept row-level data loss.		
NL-I-07	Full pg_amcheck sweep	pg_amcheck --all --heapallindexed post-recovery	Automatic	Sweep test — pg_amcheck --all --heapallindexed post-recovery. No recovery procedure — this IS the verification step. Run after every Tier-1 and Tier-2 scenario.	errors == 0 after every Tier-1 and Tier-2 scenario	**P0**
NL-I-08	Logical invariant validation	Application-level FK / sum / count invariants post-recovery	Automatic	Application-level invariant check post-recovery. FK/sum/count checks — no recovery procedure; this IS verification.	invariant_violations == 0	**P0**
NL-I-09	Checksum-disabled detection	Run against a cluster with data_checksums=off	Automatic	Detection test. Framework refuses to certify without checksums.	The framework **refuses to certify** and reports the gap	**P1**

				No recovery — the refusal IS the correct behaviour.		

10.5 NL-R · Resource exhaustion

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri
NL-R-01	OOM kill of a backend	cgroup limit; trigger the OOM killer on a backend	Automatic	1. OOM kills a backend process. 2. Postmaster detects; terminates all backends; enters recovery. 3. WAL replay; database reopens.	Postmaster survives or restarts; rpo_txn == 0; rto_first_write_s ≤ 60	**P0**
NL-R-02	OOM kill of the postmaster	OOM killer targets the postmaster	Semi Automatic	1. OOM kills the postmaster. 2. systemd Restart=always triggers restart.	Recovery is unattended; corruption_count == 0	**P0**

				3. WAL replay; database reopens. TESTER: Verify OOM target in dmesg. Record which process was killed.		
NL-R-03	Data filesystem full	fallocate to 100% under load	Manual Operation	1. Data filesystem at 100%. Writes rejected. 2. Database may PANIC if WAL also affected. RECOVERY: Free disk space (delete old logs, WAL, temp files). If PG still running: resumes when space freed.	Writes rejected with an explicit error; reads continue; no corruption; recovers when space is freed	**P0**

				If PG PANICed: restart after freeing space.		
NL-R-04	Connection exhaustion	Open max_connections + 50%	Automatic	Self-recovering once clients disconnect. Use pg_terminate_backend() via superuser to clear idle sessions.	Rejections are explicit; superuser reserved slots honoured; existing sessions unaffected	**P1**
NL-R-05	File descriptor exhaustion	Low ulimit -n, touch many relations	Semi Automatic	FD exhaustion prevents new file opens. RECOVERY: Close idle connections; increase ulimit; restart if needed.	Explicit error; no crash; no corruption	**P1**
NL-R-06	CPU saturation from our own workload	stress-ng --cpu to 100% for 120 s	Automatic	CPU at 100% from own workload. No crash.	No backend crash; checkpoint completes; WAL writes continue; p99 ≤ 5 × baseline and degradation	**P0**

				Self-recovering once workload pressure reduces.	attributed to CPU in telemetry. *Failover suppression under this condition is asserted by CL-F-07, not here — this scenario stays node-local.*	
NL-R-07	I/O saturation	fio saturating the data volume	Automatic	I/O saturation from fio. p99 degrades but no crash. Self-recovering when fio stops.	p99 degrades but there is no crash; no false failover; checkpoint completes	**P1**
NL-R-08	Temp space exhaustion	Fill temp_tablespaces with a large sort	Automatic	Temp space full. Offending query fails; others continue.	The offending query fails; no backend crash; other sessions sustain ≥ 95% baseline TPS	**P2**
NL-R-09	work_mem memory pressure	Many concurrent hash joins at high work_mem	Automatic	work_mem pressure. Spill to disk or explicit failure.	Spill to disk or explicit failure; no OOM cascade	**P1**

10.6 NL-N · Noisy neighbour and contention

Why this is a separate category from NL-R

NL-R injects resource pressure **on the target node, under our control**. Noisy neighbour is **external, invisible from inside the guest, and usually partial and sustained** rather than a clean failure. The signatures are entirely different:

- **CPU steal** — the guest sees *low* load and *high* latency at the same time. Nothing inside the database looks busy. Operators and health probes both misread it.
- **cgroup throttling** — bursty stalls at CFS period boundaries, not sustained saturation. Average utilisation can look healthy while p99 collapses.
- **Burst-credit depletion** — a cloud volume performs to specification for hours, then falls off a cliff with no configuration change and no error.

This category also covers **grey failure**: sustained partial degradation that never crosses a threshold, is never alerted, and is the hardest failure class to detect. It is also the most common real-world *trigger* for the spurious failover tested in CL-F-07 — that scenario tests the consequence; these test the cause.

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri
NL-N-01	Hypervisor CPU steal	A competing VM saturating the host; measure guest %st	Automatic	1. External contention injected (CPU steal, cgroup throttle, IOPS exhaust, memory balloon, bandwidth, backup, analytics,	Steal correctly identified as steal, not misdiagnosed as database load; leader_changed == false; p99 degradation < 3× baseline	**P0**

				grey degradation). 2. Database degrades gracefully — no crash, no failover. 3. Contention clears. Performance returns to baseline. 4. No recovery action needed by tester or operator. KEY VALIDATION: leader_changed == false throughout. If the cluster fails over during contention, that is a CL-F-07 failure, not a		

				recovery issue.		
NL-N-02	cgroup CPU throttling	systemctl set-property postgresql.service CPUQuota=50% — no container required	Automatic	1. External contention injected (CPU steal, cgroup throttle, IOPS exhaust, memory balloon, bandwidth, backup, analytics, grey degradation). 2. Database degrades gracefully — no crash, no failover. 3. Contention clears. Performance returns to baseline. 4. No recovery action needed	cpu.stat throttling counters exposed and alerted (nr_throttled plus throttled_usec on cgroup v2, throttled_time on v1); no false health-probe timeout; leader_changed == false	**P0**

				by tester or operator. KEY VALIDATION: leader_changed == false throughout. If the cluster fails over during contention, that is a CL-F-07 failure, not a recovery issue.		
NL-N-03	Shared storage IOPS exhaustion	A co-tenant saturates shared SAN/EBS; or drive burst-credit depletion to zero	Automatic	1. External contention injected (CPU steal, cgroup throttle, IOPS exhaust, memory balloon, bandwidth, backup, analytics, grey	fsync p99 ≤ 10 × baseline and attributed to storage in telemetry; checkpoint completes; WAL stall alerted; rpo_txn == 0; no spurious failover	**P0**

				degradation). 2. Database degrades gracefully — no crash, no failover. 3. Contention clears. Performance returns to baseline. 4. No recovery action needed by tester or operator. KEY VALIDATION: leader_changed == false throughout. If the cluster fails over during contention, that is a CL-F-07 failure, not a		

				recovery issue.		
NL-N-04	Host memory pressure / ballooning	Hypervisor reclaims guest memory; cgroup reclaim without OOM	Automatic	1. External contention injected (CPU steal, cgroup throttle, IOPS exhaust, memory balloon, bandwidth, backup, analytics, grey degradation). 2. Database degrades gracefully — no crash, no failover. 3. Contention clears. Performance returns to baseline. 4. No recovery action needed	No OOM kill of the postmaster; shared_buffers resident set ≥ 50% of the configured size; TPS ≥ 50% baseline; degradation detected	**P1**

				by tester or operator. KEY VALIDATION: leader_changed == false throughout. If the cluster fails over during contention, that is a CL-F-07 failure, not a recovery issue.		
NL-N-05	Network bandwidth contention	A co-tenant saturates the shared NIC	Automatic	1. External contention injected (CPU steal, cgroup throttle, IOPS exhaust, memory balloon, bandwidth, backup, analytics, grey	replication_lag_s ≤ 60 during contention, returning to ≤ 1 s within 60 s of it clearing; backpressure applied; leader_changed == false	**P1**

				degradation). 2. Database degrades gracefully — no crash, no failover. 3. Contention clears. Performance returns to baseline. 4. No recovery action needed by tester or operator. KEY VALIDATION: leader_changed == false throughout. If the cluster fails over during contention, that is a CL-F-07 failure, not a		

				recovery issue.		
NL-N-06	Internal — backup contending with peak OLTP	pgBackRest full backup on the same host and volume during peak load	Automatic	1. External contention injected (CPU steal, cgroup throttle, IOPS exhaust, memory balloon, bandwidth, backup, analytics, grey degradation). 2. Database degrades gracefully — no crash, no failover. 3. Contention clears. Performance returns to baseline. 4. No recovery action needed	OLTP p99 impact < 20%; backup completes; no failover triggered	**P1**

				by tester or operator. KEY VALIDATION: leader_changed == false throughout. If the cluster fails over during contention, that is a CL-F-07 failure, not a recovery issue.		
NL-N-07	Internal — analytics starving OLTP	A long analytic query with large sorts evicting shared buffers	Automatic	1. External contention injected (CPU steal, cgroup throttle, IOPS exhaust, memory balloon, bandwidth, backup, analytics, grey	OLTP TPS ≥ 70% baseline and p99 ≤ 2 × baseline; buffer cache hit ratio ≥ 80% of baseline; resource limits enforced if configured	**P1**

				degradation). 2. Database degrades gracefully — no crash, no failover. 3. Contention clears. Performance returns to baseline. 4. No recovery action needed by tester or operator. KEY VALIDATION: leader_changed == false throughout. If the cluster fails over during contention, that is a CL-F-07 failure, not a		

				recovery issue.		
NL-N-08	Sustained grey degradation	Reduce effective capacity by 20% for 4 h — no clean failure at any point	Automatic	1. External contention injected (CPU steal, cgroup throttle, IOPS exhaust, memory balloon, bandwidth, backup, analytics, grey degradation). 2. Database degrades gracefully — no crash, no failover. 3. Contention clears. Performance returns to baseline. 4. No recovery action needed	**Detected and alerted as degradation** — neither silence nor spurious failover; degraded-mode SLO met, or degraded state explicitly declared	**P0**

				by tester or operator. KEY VALIDATION: leader_changed == false throughout. If the cluster fails over during contention, that is a CL-F-07 failure, not a recovery issue.		

Cross-tier assertion

The chain *noisy neighbour → health-probe timeout → spurious failover → RPO risk* is the highest-value linkage in the framework. Every NL-N scenario asserts leader_changed == false when executed against a clustered target, and NL-N-01/02/03 are the realistic fault sources for CL-F-07.

10.7 NL-M · Maintenance, vacuum and wraparound

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri
NL-M-01	XID wraparound protective shutdown	pg_resetwal -x on a scratch cluster to	Manual Operation	XID/multixact wraparound protective shutdown.	Warnings escalate at documented thresholds; recovery restores	**P0**

		advance the XID counter close to the threshold, then drive a normal workload across it. *(Organic wraparound needs ~2.1 bn transactions and is not executable in a test window.)*		RECOVERY: Vacuum the offending database in single-user mode: postgres --single -D $PGDATA <dbname> VACUUM FREEZE; Then restart in normal mode.	service; no data loss	
NL-M-02	Multixact wraparound	pg_resetwal -m to advance the multixact counter on a scratch cluster, then generate multixacts via concurrent SELECT … FOR SHARE across the boundary	Manual Operation	XID/multixact wraparound protective shutdown. RECOVERY: Vacuum the offending database in single-user mode: postgres --single -D $PGDATA <dbname> VACUUM FREEZE; Then restart in normal mode.	Detected and vacuumed; protective shutdown is recoverable	**P0**

NL-M-03	Autovacuum worker killed	kill -9 an autovacuum worker repeatedly	Automatic	Autovacuum killed/starved/blocked. Self-recovering: new workers spawn; vacuum resumes. idle_in_transaction_session_timeout enforces cleanup.	A new worker spawns; no relation is left permanently unvacuumed	**P1**
NL-M-04	Autovacuum starvation	Sustained write load exceeding vacuum capacity for 4 h	Automatic	Autovacuum killed/starved/blocked. Self-recovering: new workers spawn; vacuum resumes. idle_in_transaction_session_timeout enforces cleanup.	Dead-tuple ratio ≤ 20%; TPS degradation < 30%; alerts fire before the critical threshold	**P1**
NL-M-05	Idle-in-transaction blocking vacuum	An idle transaction held for 2 h under write load	Automatic	Autovacuum killed/starved/blocked. Self-recovering: new workers	idle_in_transaction_session_timeout enforced, or a bloat alert fires	**P1**

				spawn; vacuum resumes. idle_in_transaction_session_timeout enforces cleanup.		
NL-M-06	Rolling minor-version restart	Restart under load with -m fast	Semi Automatic	Rolling restart or SIGHUP reload under load. Self-recovering after restart completes.	Availability gap ≤ 5 s per node; rpo_txn == 0	**P1**
NL-M-07	Config reload under load	SIGHUP with changed parameters at 1000 TPS	Semi Automatic	Rolling restart or SIGHUP reload under load. Self-recovering after restart completes.	Zero dropped connections; zero failed transactions	**P1**
NL-M-08	Clock skew and NTP loss	±5 min offset; stop chronyd	Automaitc	Clock skew. No crash. Self-recovering.	No backend crash; TPS ≥ 95% baseline; skew detected and reported within 300 s	**P2**

Tier 1 total: 61 scenarios — P0 36 · P1 23 · P2 2

11. Tier 2 — Cluster Scenarios (CL)

Applies to any replicated HA group — the standalone Patroni cluster, the DistDB Primary Coordinator and standby pair, and each DistDB 4-node worker cluster.

Applies key: S = standalone Patroni cluster · PC = DistDB Primary Coordinator plus standby · W = each DistDB worker cluster · all = every node type including Query Coordinators.

11.1 CL-F · Failover and promotion

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri	Applies
CL-F-01	Graceful switchover	Patroni switchover	Automaitc	1. Patroni switchover initiated. 2. Old primary demotes; new primary promotes. 3. PgBouncer reroutes connections. 4. Fully automated — no tester action	rpo_txn == 0; rto_first_write_s ≤ 30; rto_to_slo_s ≤ 120	**P0**	S, PC, W
CL-F-02	Ungraceful primary failure	kill -9 the primary	Automaitc	1. Primary killed (kill -9) or VM force-off. 2. Patroni detects within ttl; triggers election. 3. Sync standby promoted; old primary fenced. 4. PgBouncer reroutes. Clients reconnect.	rpo_txn == 0; rto ≤ 60; full RTO decomposition recorded; **Elle reports no G0/G1a/G1b/G1c/G-**	**P0**	S, PC, W

				5. Full RTO decomposition: T_detect + T_elect + T_promote + T_reconnect.	**single/G2 anomalies** in the recorded history at the declared isolation level		
CL-F-03	Primary host power loss	VM force-off	Automaitc	1. Primary killed (kill -9) or VM force-off. 2. Patroni detects within ttl; triggers election. 3. Sync standby promoted; old primary fenced. 4. PgBouncer reroutes. Clients reconnect. 5. Full RTO decomposition: T_detect + T_elect + T_promote + T_reconnect.	rpo_txn == 0; rto ≤ 60; **Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies** in the recorded history at the declared isolation level	**P0**	S, PC, W
CL-F-04	Cascading failover	Fail the primary, then fail the new primary 60 s later	Automaitc	1. First failover completes (A recovery). 2. New primary killed 60s later. 3. Second failover from remaining	The second failover succeeds; rpo_txn == 0	**P1**	S, PC, W

				replica. Both failovers are Automatic via Patroni.	both times		
CL-F-05	Failback via pg_rewind	Restart the old primary after failover	Semi Automatic	1. Old primary restarted after failover. 2. pg_rewind synchronises it with the new timeline. 3. Rejoins as replica automatically. If pg_rewind fails: see CL-F-06 (escalate to MO)	Rejoins as a replica automatically; data matches the new primary exactly	**P1**	S, PC, W
CL-F-06	pg_rewind refusal / timeline divergence	Old primary with diverged WAL after a partition	Manual Operation	1. Old primary has diverged WAL after partition. 2. pg_rewind REFUSES rather than corrupting. RECOVERY: Rebuild replica from scratch via pg_basebackup from new primary. Divergent transactions quantified.	Rewind refuses rather than corrupting; the documented reconciliation succeeds; divergent transactions quantified	**P0**	S, PC, W

CL-F-07	Failover suppression — transient disturbance	Network drop shorter than ttl; CPU stall; slow disk	Automaitc	SUPPRESSION test — no recovery needed. The system must NOT fail over during transient disturbance.	leader_changed == false for every sub-threshold disturbance	**P0**	S, PC, W
CL-F-08	RTO budget decomposition	Standard kill -9, fully instrumented	Automaitc	Standard kill -9, fully instrumented for RTO decomposition. Same recovery as CL-F-02.	Each of T_detect / T_elect / T_promote / T_reconnect within its own budget	**P0**	S, PC, W
CL-F-09	Cold-cache performance cliff	Measure the post-promotion throughput recovery curve	Automaitc	Cold-cache performance measurement after promotion. No recovery action — measures the warm-up curve.	rto_to_slo_s ≤ 180 at ≥ 80% baseline TPS, p99 ≤ 1.5× baseline	**P1**	S, PC, W
CL-F-10	Promotion with a lagging replica	Induce 60 s of lag, then fail the primary	Automaitc	1. Lagging replica + primary kill. 2. If lagged replica NOT promoted: correct (A). 3. If promoted: RPO measured and reported honestly.	Either the lagged replica is not promoted, or RPO is measured and	**P0**	S, PC, W

					reported honestly		

11.2 CL-S · Split-brain and fencing

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri	Applies
CL-S-01	Primary isolated from the consensus store	Block primary → etcd	Automaitc	1. Partition injected; primary loses consensus store or peer visibility. 2. Primary self-demotes within ttl (Patroni watchdog/softdog). 3. Remaining side elects new primary. 4. On partition heal: old primary rejoins as replica (pg_rewind). Fully automated via Patroni fencing.	The primary demotes itself within ttl; dual_write_detected == false; **Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies** in the recorded history at the declared isolation level	**P0**	S, PC, W
CL-S-02	Full partition with writers on both sides	Symmetric partition, load applied to both sides	Automaitc	1. Partition injected; primary loses consensus store or peer visibility. 2. Primary self-demotes within ttl (Patroni	Exactly one side accepts writes; the other rejects; **Elle reports no G0/G1a/G1b/G1c/G-single/G2**	**P0**	S, PC, W

				watchdog/softdog). 3. Remaining side elects new primary. 4. On partition heal: old primary rejoins as replica (pg_rewind). Fully automated via Patroni fencing.	**anomalies** in the recorded history at the declared isolation level		
CL-S-03	Fencing validation (watchdog / STONITH)	Partial failure, old primary still running. *Patroni fences via a watchdog (softdog) plus self-demotion rather than classic STONITH — test whichever mechanism is deployed*	Automaitc	1. Partition injected; primary loses consensus store or peer visibility. 2. Primary self-demotes within ttl (Patroni watchdog/softdog). 3. Remaining side elects new primary. 4. On partition heal: old primary rejoins as replica (pg_rewind). Fully automated via Patroni fencing.	The old primary cannot accept writes once the new primary is promoted	**P0**	S, PC, W
CL-S-04	Consensus store	Stop the majority	Automaitc	1. etcd majority lost. Cluster	The cluster degrades to read-only; no	**P0**	S, PC, W

	quorum loss	of etcd nodes		degrades to read-only. 2. No promotion possible without quorum. 3. RECOVERY: Restore etcd quorum (restart failed etcd members). 4. Cluster resumes normal operation.	promotion; recovers when quorum returns		
CL-S-05	Consensus store total loss	Stop all etcd nodes	Manual opereation	1. ALL etcd nodes stopped. Complete consensus loss. 2. Cluster cannot elect; current primary behaviour depends on config. RECOVERY: Restore etcd cluster (from etcd snapshot or fresh bootstrap). Restart Patroni on all nodes. Verify state convergence.	Behaviour is documented; no split-brain; the recovery procedure is validated	**P1**	S, PC, W
CL-S-06	Asymmetric partition	Primary → replica blocked, replica →	Automaitc	Asymmetric partition. Self-recovering on heal.	No dual-primary; state converges;	**P1**	S, PC, W

		primary open		Same recovery as CL-S-01.	**Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies** in the recorded history at the declared isolation level		

11.3 CL-R · Replication resilience

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri	Applies
CL-R-01	Replica network interruption	Block replication for 5 minutes	Automaitc	Replication resumes on network restore. Auto-reconnect.	Auto-reconnect; no full resync; lag ≤ 1 s within 60 s	**P0**	S, PC, W
CL-R-02	Replica process crash	kill -9 the replica	Automaitc	Replica restarts; replication resumes. No primary impact.	Replication resumes on restart; no impact on the primary	**P1**	S, PC, W
CL-R-03	Sync replica stall	SIGSTOP the sync standby	Automaitc	Sync standby SIGSTOPed; primary blocks per config. Self-recovers on SIGCONT.	The primary blocks per configuration; behaviour matches the documented contract exactly	**P0**	S, PC, W

CL-R-04	Sync quorum degradation matrix	ANY k (...) with k–1, then k–2 standbys available	Automaitc	Quorum degradation test. Behaviour per synchronous_standby_names config.	The availability/durability trade-off at each level matches the documented decision	**P0**	S, PC, W
CL-R-05	Slot bloat → pg_wal full → PANIC	Disconnect the slot consumer, generate WAL to exhaustion	Manual Operation	1. Slot consumer disconnected; WAL accumulates. 2. max_slot_wal_keep_size should invalidate slot before disk fills. 3. If disk fills before invalidation: free space manually (MO). 4. Drop orphaned slot; create fresh slot for consumer.	max_slot_wal_keep_size bounds growth; the slot is invalidated before the disk fills; the alert precedes	**P0**	S, PC, W
CL-R-06	Cascading replica chain break	Kill the middle node of a cascade	Automatic / Semi Automatic	Cascade break, consistency validation,	Downstream recovers or fails explicitly; the	**P1**	S, PC, W

				logical slot survival, conflict, feedback matrix. All self-recovering or requiring documented procedure per scenario.	documented procedure works		
CL-R-07	Continuous consistency validation	Checksum the primary against each replica throughout	Automatic / Semi Automatic	Cascade break, consistency validation, logical slot survival, conflict, feedback matrix. All self-recovering or requiring documented procedure per scenario.	divergent_rows == 0 at every sample point	**P0**	S, PC, W
CL-R-08	Logical slot survival across failover	Fail the primary with active logical subscribers	Automatic / Semi Automatic	Cascade break, consistency validation, logical slot survival, conflict, feedback matrix.	The slot survives (PG 17 failover slots) **or** the gap is explicitly documented and mitigated	**P0**	S, PC, W

				All self-recovering or requiring documented procedure per scenario.			
CL-R-09	Logical replication conflict	Insert a conflicting row on the subscriber	Automatic / Semi Automatic	Cascade break, consistency validation, logical slot survival, conflict, feedback matrix. All self-recovering or requiring documented procedure per scenario.	Conflict detected, logged, resolution policy applied; the apply worker recovers	**P1**	S, PC, W

CL-R-10	hot_standby_feedback conflict matrix	Long replica query × primary vacuum, feedback on and off	Automatic / Semi Automatic	Cascade break, consistency validation, logical slot survival, conflict, feedback matrix. All self-recovering or requiring documented procedure per scenario.	In the configured mode exactly one cost is incurred and is bounded: with feedback *off*, replica query cancellations occur and the primary dead-tuple ratio stays ≤ 20%; with feedback *on*, zero cancellations and the primary dead-tuple ratio ≤ 40%	**P1**	S, PC, W

11.4 CL-N · Network

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri	Applies
CL-N-01	Latency injection	tc netem delay 500ms	Automaitc	Network fault injected → self-recovering on network heal	leader_changes == 0; replication_lag_s ≤ 30	**P1**	all
CL-N-02	Packet loss	tc netem loss 10%	Automaitc	Network fault injected →	No corruption; no spurious failover;	**P1**	all

				self-recovering on network heal	replication continues		
CL-N-03	Bandwidth throttle	Limit the replication link to 10% of the WAL rate	Automaitc	Network fault injected → self-recovering on network heal	replication_lag_s ≤ 300 while throttled; slot protection engages before pg_wal exceeds max_slot_wal_keep_size	**P1**	all
CL-N-04	Asymmetric latency	500 ms in one direction only	Automaitc	Network fault injected → self-recovering on network heal	leader_changes == 0 for the duration	**P2**	all
CL-N-05	DNS failure	Block resolution	Automaitc	Network fault injected → self-recovering on network heal	Zero existing connections dropped; new connection attempts fail within connect_timeout with an explicit resolution error, never hanging indefinitely	**P1**	all
CL-N-06	Intermittent flapping link	10 s up / 10 s down for 5 minutes	Automatic	Network fault injected → self-recovering on network heal	No failover storm; leader_changes ≤ 1	**P0**	S, PC, W

11.5 CL-C · Connection layer

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri	Applies
CL-C-01	Pooler process failure	kill -9 PgBouncer	Automatic	pooler queues/rejects excess, self-recovers when demand falls; postmaster stays protected behind PgBouncer	Recovery ≤ 10 s; the application reconnects; rpo_txn == 0	**P1**	all
CL-C-02	Pooler behaviour during failover	Failover with the pooler in transaction mode and transactions in flight	Automatic	systemd restarts the pooler, clients reconnect via retry logic to the same primary.	In-flight transactions fail explicitly — never silently commit twice or vanish	**P0**	all
CL-C-03	Reconnect storm	Failover with 5,000 idle clients reconnecting simultaneously	Automatic	essions recover on network heal; key assertion is `leader_changed == false` (a client-path fault must	max_connections not exhausted; recovery ≤ 60 s; no thundering-herd collapse	**P0**	all

				not trigger failover).			
CL-C-04	Pooler client-side exhaustion	Exceed max_client_conn	Semi Automatic	clear the condition via SIGHUP reload or pooler restart; the reload-vs-restart distinction is the SA/MO boundary.	Explicit rejection; no database-side impact	**P1**	all
CL-C-05	Certificate expiry under load	Expire the server certificate mid-workload	Manual Operation	operator installs a valid cert and reloads — cannot self-heal — plus a prevention note that a surprise expiry is itself a finding	Existing connections behave per the documented contract; new connections fail explicitly; the alert precedes expiry	**P2**	all

11.6 CL-B · Backup, restore and disaster recovery

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri	Applies
CL-B-01	Full backup integrity	pgBackRest full backup → restore	Manual Recovery	1. Take backup (full/incremental/ chain).	row_checksum_match == 100%;	**P0**	all

				2. Restore to target instance or PITR target. 3. Verify row checksums and amcheck. These scenarios ARE the recovery procedure validation.	amcheck_errors == 0		
CL-B-02	Point-in-time recovery accuracy	DROP TABLE, restore to T−1 s	Manual Recovery	1. Take backup (full/incremental/chain). 2. Restore to target instance or PITR target. 3. Verify row checksums and amcheck. These scenarios ARE the recovery procedure validation.	The exact target LSN is reached; the drop is absent; no over- or under-shoot	**P0**	all
CL-B-03	Incremental chain restore	Full → incr1 → incr2 → restore	Manual Recovery	1. Take backup (full/incremental/chain). 2. Restore to target instance or PITR target. 3. Verify row checksums and amcheck. These scenarios ARE the recovery	The chain restores correctly; data matches the source exactly	**P0**	all

				procedure validation.			
CL-B-04	Backup under heavy load	Backup during 2000 TPS	Manual Operation	Backup under load. Recovery = backup completes; verify consistency	The backup is internally consistent; primary p99 impact < 20%	**P1**	all
CL-B-05	Restore RTO at production volume	Restore a full-size (≥ 1 TB) backup, timed	Manual Recovery	Full-size restore timed. The restore procedure IS the test.	restore_s ≤ documented DR RTO; throughput recorded	**P0**	all
CL-B-06	Backup corruption detection	Corrupt a byte in a backup file	Manual Operation	Corrupt byte in backup; pgbackrest verify detects before restore.	pgbackrest verify detects it before a restore is attempted	**P0**	all
CL-B-07	Backup repository unavailable	Block object storage during backup, and again during restore	Manual Operation	Fix repository availability; retry backup/restore	Explicit failure; retry succeeds; no partial backup is marked valid	**P1**	all

CL-B-08	Cross-region / cross-DC restore	Restore into the DR region	Manual Operation	Cross-region restore; retention/expiry safety.	DR RTO and RPO both measured against target	**P1**	all
CL-B-09	Retention / expiry safety	Run expiry with a restore in flight	Manual Operation	Cross-region restore; retention/expiry safety.	No in-use backup is deleted; the chain is never broken	**P1**	all

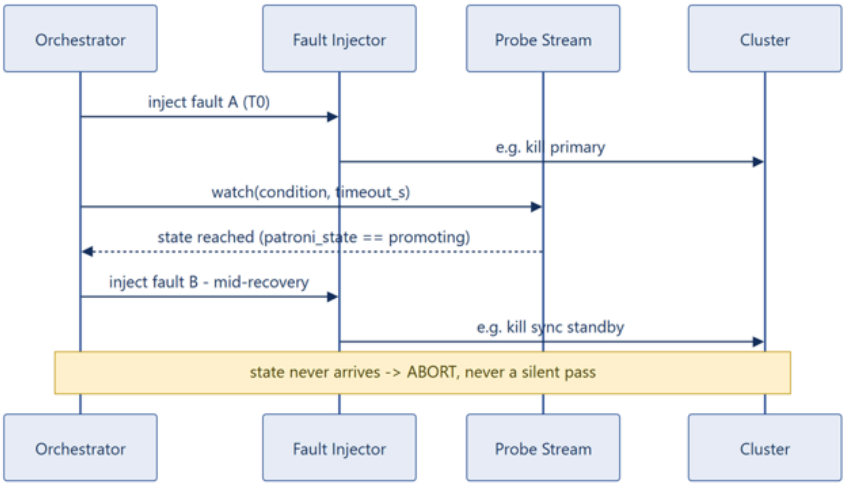
11.7 CL-X · Compound failures

Why this category exists

Every other scenario in this framework is **singular**: steady state → inject one fault → recover → validate. Production incidents are rarely singular. They look like: fault A → the system is now degraded → fault B arrives → *the recovery mechanism itself fails*.

The second fault is not injected at a fixed time. It is injected **at the moment the system is most fragile** — mid-promotion, mid-rebuild, mid-backup — which is precisely the window a singular scenario never visits.

This is a deliberately small, hand-picked suite — not a Cartesian product. Ten pairs chosen because each has a documented history of turning a recoverable incident into an outage. Combinatorial expansion of 162 scenarios would produce tens of thousands of pairs with a vanishing yield per test.



ID	Scenario	Fault A → Fault B (trigger)	Recovery Type	Recovery Procedure	Acceptance criterion	Pri	Applies
CL-X-01	Sync replica fails during promotion	Kill the primary → kill the sync standby *when Patroni enters* promoting	Semi Automatic	Primary killed + sync standby killed during promotion. RECOVERY: Cluster must converge to exactly one primary from the remaining async replica(s). If no viable replica: MO escalation — rebuild from backup	Exactly one primary after convergence; rpo_txn == 0, or loss quantified against markers and disclosed; dual_write_detected == false; recovery ≤ 2 × the single-failure RTO budget	**P0**	S, PC, W
CL-X-02	Consensus store degrades	Kill the primary → inject 2 s of	Semi Automatic	Primary killed + etcd latency	leader_changes ≤ 1; dual_write_detec	**P0**	S, PC, W

	mid-failover	etcd latency *during the election window*		during election. RECOVERY: Election completes when etcd stabilises. If cannot complete: cluster stays read-only (correct, not split).	ted == false; if the election cannot complete the cluster stays **read-only, never split**		
CL-X-03	Primary crashes with the data volume near-full	Fill the data filesystem to 95% → kill -9 the primary	Semi Automatic	Disk near-full + primary kill. RECOVERY: Failover to standby (A). If standby also near-full: free space (MO).	corruption_count == 0; recovery completes or fails explicitly — never partial replay; failover proceeds; rpo_txn == 0	**P0**	S, PC, W
CL-X-04	Primary crashes with the WAL archive unavailable	Block the archive destination → kill -9 the primary	Semi-Automatic	Archive blocked + primary kill. RECOVERY: Failover proceeds. Archive backlog drains on restore. Verify no gap.	rpo_txn == 0; pg_wal bounded per NL-W-04; the archive backlog drains on restore with **no gap in the sequence**	**P0**	S, PC, W

CL-X-05	Node restarts while partitioned	Partition a node → restart it *while still partitioned* → heal	Manual Operation	Node restarted while partitioned. RECOVERY: On partition heal, node must rejoin correctly or stay fenced. May need pg_rewind if diverged.	No split-brain at any point; the node rejoins correctly or stays fenced; divergent_rows == 0 after healing	**P0**	S, PC, W
CL-X-06	Pooler fails during failover	Kill the primary → kill PgBouncer *when promotion starts*	Semi-Automatic	Primary killed + PgBouncer killed during promotion. RECOVERY: Both restart independently. PgBouncer reconnects to new primary.	In-flight transactions fail explicitly — never silently commit twice or vanish; clients reconnect ≤ 60 s after the pooler restarts; rpo_txn == 0	**P0**	S, PC, W
CL-X-07	Primary fails mid-backup	Start a pgBackRest full backup → kill the primary *at ~50% progress*	Manual Operation	Primary killed mid-backup. RECOVERY: Failover proceeds (A). Backup marked invalid	The backup is marked invalid — **never partially valid**; failover completes within RTO; the next backup succeeds and	**P0**	S, PC, W

				(never partially valid). New backup taken from new primary.	pgbackrest verify passes on the resulting chain		
CL-X-08	A second replica fails during a rebuild	Rebuild replica A → kill replica B *mid-rebuild*	Semi-Automatic	Replica B killed during rebuild of replica A. RECOVERY: Primary sustains load. Rebuild completes or fails explicitly	The primary sustains ≥ 95% baseline TPS; the rebuild completes or fails explicitly; sync-quorum behaviour matches the documented contract (CL-R-04)	**P0**	S, PC, W
CL-X-09	Primary DC lost while DR replication is lagging	Induce cross-DC lag → stop the entire primary datacentre	Manual Recovery	Primary DC lost while DR lagging. RECOVERY: DR promotion with measured data loss. RPO measured by markers, never assumed	DR RPO **measured against markers, never assumed**; the promotion decision and its data-loss cost recorded; no split-brain across datacentres. *DistDB equivalent: DX-D-03.*	**P0**	S, PC, W

Two consequences for the harness — neither is optional

1. **Fault B is state-triggered, not time-triggered.** “Kill the sync standby 20 seconds in” is not the same test as “kill it the instant promotion begins” — and only the second reliably hits the fragile window. The orchestrator must arm the second injection on a *predicate over the probe stream*.
2. **Blast radius rises to 2 nodes for this category alone.** Every other scenario is capped at max_nodes_affected: 1. Compound scenarios are, by definition, two concurrent faults. This is a documented, category-scoped exception — **not** a relaxation of the global safety rule — and the Safety Controller must enforce 2 as the hard ceiling here and 1 everywhere else.

Tier 2 total: 55 scenarios — P0 34 · P1 19 · P2 2

12. Tier 3 — Distributed Scenarios (DX)

DistDB only. These scenarios exercise the distributed layer: 2PC (mandatory for DDL, optional for DML), 4-node worker clusters with synchronous replication, 64-bit GXIDs, and a single Primary Coordinator with no consensus protocol for metadata.

12.1 DX-T · Two-phase commit and in-doubt transactions

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri
DX-T-01	Coordinator fails after PREPARE, before COMMIT	Kill the Primary Coordinator between phases	Automatic	1. Primary Coordinator or participant fails during 2PC. 2. In-doubt transaction resolver activates on restart. 3. Resolver queries participants for prepared transaction state. 4. All participants converge to same outcome (commit or abort).	All participants converge to the same outcome; in_doubt_after_recovery == 0; **Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies** in the recorded history at the	**P0**

				CRITICAL: in_doubt_after_recovery must == 0. Orphaned prepared transactions are a P0 defect	declared isolation level	
DX-T-02	Participant fails after PREPARE	Kill a worker holding a prepared transaction	Automatic	1. Primary Coordinator or participant fails during 2PC. 2. In-doubt transaction resolver activates on restart. 3. Resolver queries participants for prepared transaction state. 4. All participants converge to same outcome (commit or abort). CRITICAL: in_doubt_after_recovery must == 0. Orphaned prepared transactions are a P0 defect	The prepared transaction survives worker restart; the resolver completes it; **Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies** in the recorded history at the declared isolation level	**P0**
DX-T-03	Coordinator fails during COMMIT broadcast	Kill the Primary Coordinator mid-broadcast	Automatic	1. Primary Coordinator or participant fails during 2PC. 2. In-doubt transaction resolver activates on restart. 3. Resolver queries participants for prepared transaction state. 4. All participants converge to same outcome (commit or abort). CRITICAL: in_doubt_after_recovery must == 0. Orphaned prepared transactions are a P0 defect	No participant diverges; a partial commit is resolved, never left split; **Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies** in the recorded history at the declared isolation level	**P0**
DX-T-	Network partition	Isolate a participant	Automati	Network partition during PREPARE. Transaction aborts	The transaction aborts cleanly	**P0**

04	during PREPARE	at prepare time	c	on all nodes or commits on all	on all nodes, or completes on all	
DX-T-05	In-doubt resolver under load	1000 concurrent distributed transactions plus a Primary Coordinator kill	Semi Automatic	1000 concurrent distributed txns + PC kill. RECOVERY: PC restarts; resolver handles all orphaned txns. May take up to 120s to resolve all	All resolved within 120 s; orphaned_prepared_txn == 0	**P0**
DX-T-06	Prepared transaction lock retention	Leave prepared transactions unresolved for 30 minutes	Manual Operation	Prepared transactions unresolved for 30 minutes. RECOVERY: Manual resolution via pg_prepared_xacts inspection + COMMIT PREPARED / ROLLBACK PREPARED per txn	No unrelated transaction waits > 60 s on a lock held by a prepared transaction; the alert fires within 300 s; no cluster-wide stall	**P0**
DX-T-07	max_prepared_transactions exhaustion	Exceed the configured limit	Automatic /Semi Automatic	Resource exhaustion, DDL 2PC, DML without 2PC, GXID wraparound, distributed deadlock. Recovery per scenario: mostly automatic via resolver, deadlock detection, and protective mechanisms.	Explicit rejection; existing prepared transactions unaffected	**P1**
DX-T-08	DDL 2PC failure mid-cluster	Kill a worker during distributed DDL	Automatic /Semi Auto	Resource exhaustion, DDL 2PC, DML without 2PC, GXID wraparound, distributed deadlock. Recovery per scenario: mostly automatic via resolver, deadlock	The schema converges cluster-wide, or rolls back cluster-wide — never partial; **Elle reports no**	**P0**

			mati c	detection, and protective mechanisms.	**G0/G1a/G1b/G1c/G-single/G2 anomalies** in the recorded history at the declared isolation level	
DX-T-09	Multi-shard DML with 2PC disabled	Fail a shard mid-write	Automatic /Semi Automatic	Resource exhaustion, DDL 2PC, DML without 2PC, GXID wraparound, distributed deadlock. Recovery per scenario: mostly automatic via resolver, deadlock detection, and protective mechanisms.	Per-shard atomicity honoured; divergence detectable and quantified; **Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies** in the recorded history at the declared isolation level	**P0**
DX-T-10	GXID exhaustion / wraparound	Pre-seed the GXID counter near the limit on a scratch cluster, then drive distributed transactions across the boundary. *Requires a DistDB-side counter-seeding*	Automatic /Semi Automatic	Resource exhaustion, DDL 2PC, DML without 2PC, GXID wraparound, distributed deadlock. Recovery per scenario: mostly automatic via resolver, deadlock detection, and protective mechanisms.	Protective behaviour is documented and recoverable	**P1**

		facility — confirm one exists.				
DX-T-11	Distributed deadlock	Cross-shard circular wait	Automatic /Semi Automatic	Resource exhaustion, DDL 2PC, DML without 2PC, GXID wraparound, distributed deadlock. Recovery per scenario: mostly automatic via resolver, deadlock detection, and protective mechanisms.	Detected within timeout; one victim aborted; the cluster continues	**P1**

12.2 DX-M · Metadata and coordinators

ID	Scenario	Recovery Type	Recovery Procedure	Fault injection	Acceptance criterion	Pri
DX-M-01	Primary Coordinator crash	Automatic	1. Primary Coordinator crashes. DML continues on QCs. 2. DDL blocks explicitly. 3. PC restarts and recovers within RTO	kill -9 the Primary Coordinator	DML continues on the Query Coordinators; DDL blocks explicitly; the Primary Coordinator recovers within RTO	**P0**
DX-M-02	Primary Coordinator failover to standby	Semi Automatic	PC failover to standby. QCs re-sync metadata.	Force Primary Coordinator promotion	Metadata intact and complete; Query Coordinators re-sync; metadata_divergence == 0	**P0**

DX-M-03	Primary Coordinator unavailable for an extended period	Automatic	PC down for 30 min. DML at ≥ 95% baseline. DDL queued	Primary Coordinator down for 30 minutes under load	DML TPS ≥ 95% baseline; queued DDL bounded by the configured limit with none silently dropped; no cache staleness beyond the documented window	**P0**
DX-M-04	Metadata cache invalidation lost	Automatic	Cache invalidation lost. Stale cache detected and corrected	Drop invalidation messages to a Query Coordinator	Stale cache detected and corrected; no wrong-shard routing	**P0**
DX-M-05	Query Coordinator crash under load	Automatic	QC crashes. Load balancer routes away. No data impact	kill -9 a Query Coordinator	In-flight queries fail explicitly; the load balancer routes away; no data impact	**P0**
DX-M-06	All Query Coordinators fail simultaneously	Semi Automatic	All QCs fail. PC absorbs load or declares outage	Kill every Query Coordinator	The Primary Coordinator absorbs load, or an explicit outage is declared; recovery is unattended	**P1**
DX-M-07	Query Coordinator	Automatic	New QC added mid-load. Syncs	Scale out mid-workload	The new Query Coordinator syncs	**P1**

	added during load		metadata before serving.		metadata before serving; no wrong results during warm-up	
DX-M-08	Metadata corruption on the Primary Coordinator	Manual Recovery	Catalog corruption on PC. Restore from backup required. RECOVERY: Restore PC from backup; QCs re-sync.	Corrupt the catalog on the Primary Coordinator	Detected; the restore-from-backup path is validated	**P0**
DX-M-09	Split-brain Primary Coordinator	Automatic	Split-brain PC prevented by fencing	Partition the Primary Coordinator from its standby; both attempt promotion	Prevented by fencing; dual_pc_detected == false	**P0**

12.3 DX-S · Shard placement and rebalancing

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri
DX-S-01	Worker cluster total loss	Kill all 4 nodes of one worker cluster	Automatic	1. Worker node holding active shard placements is killed (kill -9 / VM force-off). 2. Coordinator detects the worker is down; routing marks its placements unavailable.	Affected shards are explicitly unavailable; other shards serve; no wrong results; **Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies** in the recorded history	**P0**

				3. Shard replica on a healthy worker is promoted to active (worker-cluster failover). 4. Queries re-route to the promoted placement automatically. TESTER: Observe only. If no replica placement exists for a shard, that shard is explicitly unavailable — escalate to MO rebuild from backup and record it.	at the declared isolation level	
DX-S-02	Shard move interrupted	Kill the source mid-move	Manula Operation	1. Shard move / rebalance is interrupted mid-flight (source copied, destination partial, cutover not committed). 2. Placement metadata may reference an incomplete destination placement. RECOVERY (operator): a) Inspect placement catalog for placements in a transitional/invalid state. b) Drop the partial destination placement; confirm the source	The move rolls back or completes; shard_copies == 1; no orphan	**P0**

				placement is intact and authoritative. c) Verify row counts / checksums between source and any completed copy before trusting it. d) Re-issue the shard move once the cluster is clean		
DX-S-03	Rebalance under load	Rebalance during 1000 TPS	Automatic	1. Shard replica (standby copy of a placement) fails or falls behind. 2. Coordinator continues serving from the authoritative placement. 3. Replica placement is re-created / re-synced automatically from the authoritative copy. No data impact — the authoritative placement is untouched.	No lost writes; p99 impact < 30%; placement metadata consistent	~~**P0**~~
DX-S-04	Node addition during a failure	Add a worker while another is down	Automatic	online moves with atomic cutover, no dual-placement window.	The addition succeeds or fails cleanly; no metadata corruption	**P1**
DX-S-05	Node removal with active queries	Drain a worker with queries in flight	Automatic	cutover serialises against in-flight writes; no write lands on a stale placement	Queries complete or fail explicitly; no silent truncation of results	**P0**

DX-S-06	Reference table replication failure	Fail a worker during a reference-table 2PC write	Semi Automatic	re-sync the placement map from the workers and restart the routing cache — shards are intact, only the map is stale.	Reference tables converge on all workers, or roll back everywhere; **Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies** in the recorded history at the declared isolation level	**P0**
DX-S-07	Colocation constraint under failure	Fail a node holding colocated shards	Semi Automatic	free space, then resume/retry; falls back to the DX-S-02 cleanup path if the partial placement is unusable	Colocation is preserved after recovery	**P1**
DX-S-08	Shard co-tenancy blast radius	One shard's runaway workload saturates a worker hosting other shards			The blast radius is contained; other shards on that worker stay within SLO or are explicitly declared degraded — never silently slow	**P0**

12.4 DX-Q · Distributed query execution

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri

DX-Q-01	Worker fails mid-fragment	Kill a worker during a distributed query	Automatic	• Query failures are transient — retry or fail explicitly. No data impact. No recovery procedure beyond "wait for the failed component to restart."	The query fails explicitly or retries — **never returns partial results as complete**; **Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies** in the recorded history at the declared isolation level	**P0**
DX-Q-02	Straggler worker	SIGSTOP one worker during scatter-gather	Automatic	• Query failures are transient — retry or fail explicitly. No data impact. No recovery procedure beyond "wait for the failed component to restart."	Timeout enforced; no indefinite hang	**P0**
DX-Q-03	Repartition join with node failure	Fail a worker mid-repartition	Automatic	• Query failures are transient — retry or fail explicitly. No data impact. No recovery procedure beyond "wait for the failed component to restart."	Clean failure; temporary and intermediate state cleaned up	**P0**
DX-Q-04	Result collation failure at the coordinator	OOM the coordinator during aggregation	Automatic	• Query failures are transient — retry or fail explicitly. No data impact. No recovery procedure beyond "wait for the failed component to restart."	Explicit failure; no truncated aggregate returned	**P0**

DX-Q-05	Long analytic query during failover	Failover mid-OLAP query	Automatic	• Query failures are transient — retry or fail explicitly. No data impact. No recovery procedure beyond "wait for the failed component to restart."	The query fails explicitly; retry succeeds; no wrong answer	**P1**
DX-Q-06	Intermediate file cleanup	Kill workers repeatedly during repartition	Automatic	• Query failures are transient — retry or fail explicitly. No data impact. No recovery procedure beyond "wait for the failed component to restart."	No unbounded temporary-file accumulation	**P1**
DX-Q-07	Contended straggler amplification	One worker degraded by a noisy neighbour — slow, **not** failed (contrast DX-Q-02)	Automatic	• Query failures are transient — retry or fail explicitly. No data impact. No recovery procedure beyond "wait for the failed component to restart."	Scatter-gather latency = max(shard); degradation attributed to the correct worker, not to the query; a timeout or hedged read bounds the impact	**P1**

12.5 DX-P · Cross-node partition

ID	Scenario	Fault injection	Recovery Type	Recovery Procedure	Acceptance criterion	Pri
DX-P-01	Coordinator ↔ worker partition	Block PC/QC → a subset of workers	Automatic	Self-recovering on partition heal	Affected shards are explicitly unavailable; no stale reads served as fresh; **Elle reports no G0/G1a/G1b/G1c/G-**	**P0**

					single/G2 anomalies in the recorded history at the declared isolation level	
DX-P-02	Worker ↔ worker partition	Block inter-worker traffic	Automatic	Self-recovering on partition heal	Repartition queries fail explicitly; no wrong results; **Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies** in the recorded history at the declared isolation level	**P0**
DX-P-03	PC ↔ QC partition	Block metadata synchronisation	Automatic	Self-recovering on partition heal	Query Coordinators serve from cache within the staleness bound, then refuse	**P0**
DX-P-04	Single worker isolated	Isolate one worker node	Automatic	Self-recovering on partition heal	Its shards fail over within the worker cluster; RTO within target	**P0**
DX-P-05	Partition healing convergence	Restore all links after 10 minutes	Automatic	Self-recovering on partition heal	divergent_rows == 0; convergence completes ≤ 300 s after link restore	**P0**
DX-P-06	Partition exactly at the commit point	Partition timed to the 2PC commit boundary	Automatic	Self-recovering on partition heal	See DX-T-04; no participant divergence; **Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies** in the recorded history at the declared isolation level	**P0**

12.6 DX-D · Multi-datacentre disaster recovery

ID	Scenario	Recovery Type	Recovery Procedure	Fault injection	Acceptance criterion	Pri
DX-D-01	Primary datacenter total loss	Manual Operation	DR promotion	Stop everything in DC1	DR promotion within the DR RTO; RPO measured and within target	**P0**
DX-D-02	Cross-DC link loss	Automatic	self-recovering on link restore	Sever inter-datacentre replication for 30 minutes	No split-brain; cross-DC lag ≤ the documented DR RPO bound; catch-up completes ≤ 2 × the outage duration	**P0**
DX-D-03	DR promotion with lag	Manual Operation	promotion with measured RPO	Promote the DR site while lagging	RPO honestly measured and reported — never assumed zero	**P0**
DX-D-04	Failback to the primary datacentre	Manual Operation	data reconciliation	Return to DC1 after DR operation	Data reconciled; divergent_rows == 0; **Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies** in the recorded history at the declared isolation level	**P1**
DX-D-05	Partial datacentre failure	Semi Automatic	bring the lost nodes back to operations	Lose half of DC1's nodes	The cluster degrades predictably; no data loss	**P1**

Tier 3 total: 46 scenarios — P0 34 · P1 12

13. Design Standards

These standards define what counts as a valid resilience test in this programme. Each one changes what must be built, and in what order.

13.1 Durability cannot be validated by process kill

Killing the postmaster terminates the process but leaves the OS page cache fully intact — the kernel still writes those dirty pages to disk afterwards. Such a test exercises PostgreSQL's WAL replay path and **nothing below it**.

Configurations that pass a kill -9 test every time while losing committed data in a real power event: fsync=off · synchronous_commit=off · a volatile write-back controller cache without BBU · a hypervisor in cache=unsafe or writeback mode · volume drivers that ignore fdatasync · incorrectly mounted network storage.

Method	What it proves
pg_test_fsync baseline	Whether flushes are plausibly real — suspiciously high ops/sec indicates a lying cache
VM hard power-off (virsh destroy, cloud force-stop)	Hypervisor and guest cache honesty
dm-flakey / dm-error device-mapper injection	Block-layer write loss and torn writes
Physical power cut on a lab node	Ground truth, including the controller cache

13.2 Acceptance criteria must be falsifiable

Phrases such as "graceful degradation", "no crash", "system functional", "controlled behaviour", "documented behaviour" and "recovery within SLA" (with no number given) cannot gate a build. Every criterion in this framework is a **numeric or boolean predicate**, and this is enforced mechanically (§13.7, check 3).

13.3 Failover suppression is a first-class requirement

Testing only that failover *happens* is half the requirement. In production, spurious failover triggered by a transient network blip, a CPU starvation stall, a long checkpoint or a slow disk is a leading cause of avoidable outage — and each spurious failover carries real RPO risk. **CL-F-07, CL-N-06 and NL-R-06** assert the negative: that the cluster stays put when it should.

13.4 Noisy neighbour and grey failure are a distinct class

Self-injected resource pressure on the node under test is a fundamentally different failure signature from external contention on shared infrastructure, and it misses the hardest classes entirely:

Mechanism	Why stress-ng does not reproduce it
Hypervisor CPU steal	The guest sees *low* load with *high* latency. Nothing inside the database looks busy, so both operators and health probes misdiagnose it.
cgroup CPU throttling	Bursty stalls at CFS period boundaries. Average utilisation looks healthy while p99 collapses.
Burst-credit depletion	The volume performs to specification for hours, then falls off a cliff — no configuration change, no error, no event.
Grey failure	Sustained partial degradation that never crosses a threshold, never alerts, and is never attributed to the right cause.

Category **NL-N** (8 scenarios) plus **DX-S-08** and **DX-Q-07** cover this class. They also supply the realistic *cause* for the contention-induced spurious failover tested in CL-F-07.

13.5 Consistency checking must reach transaction-isolation anomalies

Predicates such as invariant_violations == 0, divergent_rows == 0 and "divergence detectable and quantified" detect **gross divergence only**. None of them can detect a **transaction-isolation anomaly** — lost update, read skew, write skew, aborted read (G1a), intermediate read (G1b), or a dependency cycle (G1c/G2).

For a distributed database in which **2PC is optional for DML**, that would be a material gap. DX-T-09 is precisely the scenario most likely to produce isolation anomalies.

> **Standard**
>
> **18 scenarios additionally require Elle** — the transactional-anomaly checker from the Jepsen project — to report no G0/G1a/G1b/G1c/G-single/G2 anomalies in the recorded operation history at the declared

isolation level. This is an *additional* predicate, not a replacement: the existing timing and integrity criteria still apply.

Scenarios: NL-C-03 · CL-F-02/03 · CL-S-01/02/06 · DX-T-01/02/03/08/09 · DX-P-01/02/06 · DX-Q-01 · DX-S-01/06 · DX-D-04.

Elle is adopted as a **checker over an exported history**, not as a test framework. The harness stays in Python.

13.6 One machine-readable source of truth

The catalog lives in **one YAML/CSV file** from which every document — this one included — is generated. Hand-maintaining parallel Markdown, HTML and CSV copies produces silent divergence, and divergence in a compliance artefact is a defect.

13.7 Catalog validation — checks applied

Subjective read-throughs surface a different set of defects on every pass, so review never converges. The checks below are **mechanical and repeatable**, and are re-run against the machine-readable catalog on every change.

#	Check	Rule	Last result
1	Row well-formedness	Every scenario row carries ID, name, fault, criterion and priority (plus *applies* where the table defines it)	162 / 162 pass
2	ID contiguity	Each category numbered 1..n with no gaps or duplicates	21 categories, no gaps
3	**Criterion falsifiability**	Every acceptance criterion contains a numeric	162 / 162 pass

		threshold or a boolean predicate; no vague term may be the *operative* assertion	
4	Scenario cross-references	Every referenced scenario ID is defined somewhere in the catalog	No dangling references
5	Section cross-references	Every §n.n reference resolves to an existing heading	All resolve
6	Numeric reconciliation	Tier totals, category counts, priority splits, the coverage matrix and all derived arithmetic agree with the row data	All reconcile

On the thresholds introduced by check 3

Values such as “p99 ≤ 5 × baseline” or “dead-tuple ratio ≤ 20%” are **initial defaults chosen to make each criterion testable**. They are deliberately expressed relative to measured baseline where possible, so that they port across environment classes. Consistent with open decision 5 (§19), they must be **re-baselined against measurement** before being treated as contractual.

14. Scope Boundaries and Deferred Items

The following are known and consciously deprioritised for this release. They are recorded so that they are not later rediscovered as incidents, and are revisited at each quarterly review.

Deferred item	
Cluster-consistent backup and restore for DistDB (no DX-B category)	CL-B is per-node. Restoring shards to different points yields a silently inconsistent database; cluster-wide PITR across coordinator metadata and N worker clusters is untested.
The HA agent (Patroni) as its own failure domain	If the agent dies while PostgreSQL is healthy, the leader key expires and another node promotes while the old primary still accepts writes — with nothing alive to demote it. This is the failure the Patroni watchdog exists to prevent.
Delayed standby (recovery_min_apply_delay)	A deployed protection mechanism against logical corruption, not yet validated.
Monitoring and detection stack failure	Detectability is one of the five properties in §2.2 and currently has no failure scenario. If the exporter dies, MTTD is unbounded.
Distributed rolling upgrade / mixed-version cluster	NL-M-06 covers single-node minor restart only.
Lower-severity items	Prepared transactions across failover on standalone PostgreSQL · lock-table exhaustion · inode exhaustion · shared-memory and semaphore limits blocking restart · auth-backend outage and emergency access · an orphaned slot on a promoted primary · a

	scale_sensitive axis to validate SLA targets at production data volume.

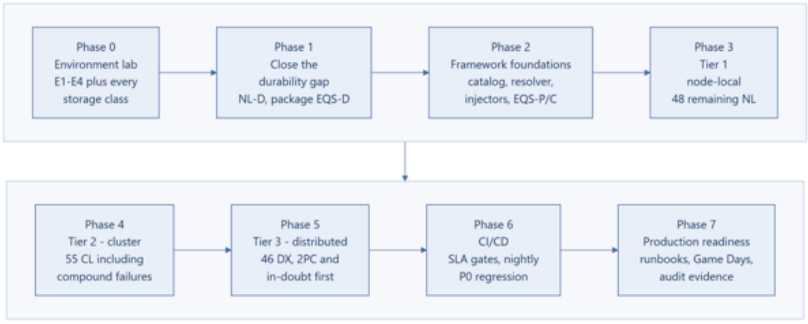


Figure 11 — Delivery phases. Durability is closed before the framework is built.

Phase	Focus	Content	Exit gate
Phase 0	Environment lab	Stand up a lab representation of every supported class: E1 bare metal, E2 dedicated-host VMs (with a co-tenant VM for NL-N), E3 shared cloud, E4 burstable. Every storage class to be certified is represented alongside.	Every supported environment class is reproducible in the lab
Phase 1	Close the durability gap	NL-D-01 … NL-D-06 across every certified storage class. Power-loss harness, dm-flakey rig, fsync baselines,	The RPO = 0 claim is *evidenced* per storage class; storage classes that cannot support it are identified

		synchronous_commit matrix. **Package as EQS-D.**	
Phase 2	Framework foundations and qualification suite	Machine-readable catalog plus document generation; target resolver with a DistDB role strategy; **SSH/systemd/`tc`/`dm-*` injection adapters**; RTO decomposition instrumentation; transaction-marker library; **EQS-P and EQS-C packaged with report generation**	One Tier-1 scenario runs end-to-end against all node roles; EQS is runnable against an arbitrary customer environment
Phase 3	Tier 1 — node-local	The remaining 48 NL scenarios, P0 first, executed against standalone and all DistDB roles. **NL-N requires a co-tenant load source** — a second VM on the same lab host (E2). It	All Tier-1 P0 scenarios pass on both systems

		cannot be injected on E3.	
Phase 4	Tier 2 — cluster	All 55 CL scenarios, including the compound category, against the standalone Patroni cluster, the Primary Coordinator pair, and the worker clusters	All Tier-2 P0 scenarios pass; the RTO budget is decomposed and within target
Phase 5	Tier 3 — distributed	All 46 DX scenarios; 2PC and in-doubt resolution first	All Tier-3 P0 scenarios pass
Phase 6	CI/CD and continuous validation	Jenkins pipeline with SLA gates; JUnit output; nightly P0 regression; Grafana dashboards	The nightly run is green; regressions are detected automatically
Phase 7	Production readiness	Runbook validation; Game Day exercises; audit evidence package; team training	Sign-off

Sequencing rationale

Phase 1 precedes framework work deliberately. Until the durability gap is closed, every other result is reported against an unverified foundation, and

no RPO = 0 statement can be supported. It is the highest-value week of work in the programme.

Phase 2 precedes bulk scenario work because the target resolver is what makes Tier-1 reuse across DistDB node roles possible. Building scenarios first would mean rewriting them.

16. Deliverables, Entry and Exit Criteria

16.1 Deliverables

Deliverable	Description	Audience	Frequency
Test Execution Report	Per-scenario results with all measured metrics	Engineering	Per run
SLA Compliance Report	RTO decomposition, RPO, integrity against targets, both systems	Leadership	Weekly
Durability Evidence Package	NL-D results per storage class, fsync baselines, power-loss logs	Auditors, Leadership	Quarterly
Environment Qualification Report	Per deployment: measured EQS-D/P/C results, assigned SLA tier, disclosed limitations	**Customer**, Field Engineering, Sales	Per deployment; re-run on infrastructure change
Supported Environment Matrix	Which environment and storage classes are certified for which SLA tier	Product, Sales, Support	Per release

Trend Analysis	Metrics over time, regression detection	All	Monthly
Incident Analysis	Root cause for any failed scenario	Engineering	As needed
Audit Evidence Package	Full compliance documentation with raw artefacts	Auditors	Quarterly

16.2 Entry criteria

- Test environment provisioned, isolated from production, and healthy
- Target cluster running with replication configured and healthy
- data_checksums enabled — the framework refuses to certify without it (NL-I-09)
- Monitoring, alerting and log aggregation operational
- Backup system operational, with at least one verified restore
- Steady-state baseline captured and stable for the declared duration
- Power-loss injection capability available for NL-D (hypervisor force-off at minimum)
- Co-tenant load source available for NL-N on E2 lab hosts — a second VM we control on the same physical host — plus visibility of %st, nr_throttled and storage burst-credit metrics
- Lab representation of every supported environment class (E1, E2, E3, E4) for the environment-sensitive scenario subset
- TDE enabled with a reachable, test-safe KMS/keystore — NL-E cannot run without it

16.3 Exit criteria

- **All 104 P0 scenarios pass** on their applicable systems
- All P1 scenarios pass, or carry documented and accepted remediation

- **The NL-D category passes** — durability is evidenced, not assumed
- RTO within target *with the decomposition recorded*, and RTO-to-SLO within target
- RPO = 0 validated by transaction markers under power loss, not process kill
- Zero split-brain conditions observed across all CL-S scenarios
- Zero spurious failovers across all suppression scenarios
- pg_amcheck clean after every Tier-1 and Tier-2 scenario
- Test report reviewed and approved by the named signatories

17. Risk Register

Risk	Impact	Likelihood	Mitigation and validating scenarios
Committed data loss from a dishonest storage stack	**Critical**	Medium	**Currently unmitigated and unmeasured.** NL-D-01 … NL-D-06
Committed data loss from misconfiguration	**Critical**	Low	synchronous_commit=on, fsync=on; NL-D-04, NL-W-01
Split-brain	**Critical**	Low	Patroni fencing; CL-S-01 … CL-S-06, DX-M-09
Undetected corruption	**Critical**	Low	Data checksums, pg_amcheck; NL-I-01 … NL-I-09

Spurious failover causing a self-inflicted outage	**High**	Medium	CL-F-07, CL-N-06, NL-R-06; realistic triggers in NL-N-01/02/03
Grey failure — sustained partial degradation never detected	**High**	**High**	**Currently unmitigated and unmeasured.** NL-N-08, NL-N-01 … NL-N-05
Noisy neighbour degrading a shared-infrastructure node	**High**	**High**	NL-N-01 … NL-N-07; DX-S-08 and DX-Q-07 for shard co-tenancy
Customer deploys on storage that discards flushes	**Critical**	**Medium**	EQS-D at go-live; Unqualified tier with written disclosure. **Outside our control — only detectable, not preventable.**
SLA committed without environment qualification	**Critical**	Medium	Process control: tier assignment must derive from an EQS report, never from a contract negotiation
Fault-injection tooling assumes containers that do not exist	**High**	**Realised**	SSH/systemd/tc/dm-* adapters built first (§7.4); container-native

			adapters deferred
Failover exceeds RTO	**High**	Medium	Patroni tuning informed by the CL-F-08 decomposition
Post-failover reconnect storm extends the outage	**High**	Medium	CL-C-02, CL-C-03
Replication slot bloat fills the primary's disk	**High**	Medium	max_slot_wal_keep_size; CL-R-05
Backup restore fails or exceeds DR RTO	**High**	Low	CL-B-01 … CL-B-09, especially CL-B-05 at production volume
2PC in-doubt transactions block the cluster	**Critical**	Medium	DX-T-01 … DX-T-06
Distributed query returns partial results as complete	**Critical**	Low	DX-Q-01, DX-Q-04, DX-P-01
XID or multixact wraparound outage	**Critical**	Low	NL-M-01, NL-M-02, NL-M-04

Testing risk

All resilience tests execute in isolated, non-production environments. Every scenario declares a blast radius and abort conditions. Production

chaos testing requires explicit approval and uses canary patterns with a limited blast radius. The Safety Controller enforces a kill switch, a circuit breaker, and a maximum-affected-nodes cap on every run.

18. Constraints Already Settled

These decisions are made and are treated as fixed inputs to the framework. They are recorded because each one visibly shapes the design.

Constraint	What it drives
Both shared and dedicated infrastructure must be supported — the customer chooses.	§3.4 environment classes; §3.5 the conditional support contract; §3.6 the Environment Qualification Suite; §6.1 the tiered SLA. Noisy neighbor is a routine production condition, not an edge case.
The database is not containerized.	§7.4 the non-containerized injection toolchain: chaosd and direct OS tooling first, container-native injectors deferred behind the same adapter interface. Environment class E5 is out of scope until containerization exists.
DistDB implements two-phase commit — mandatory for DDL and metadata, optional for DML.	The whole of Tier 3, and in particular DX-T. Any scenario premised on “partial commit without 2PC” would be invalid for this system.

19. Open Decisions Required

These cannot be resolved from existing documentation and materially affect the design. Each requires an owner and a decision before the corresponding phase begins.

#	Decision	Why it matters	Blocks

1	**Is RPO = 0 actually required, and is the availability cost accepted?**	RPO = 0 requires synchronous_commit = on or remote_apply with a healthy sync standby — which means **the primary blocks when sync quorum is lost**. That is a deliberate availability trade-off that has never been explicitly accepted in writing. NL-D-04 and CL-R-04 exist to force this into the open.	Phase 1
2	**Is a lab node available where physical power can be cut?**	If not, NL-D-02 and NL-D-06 rely on hypervisor force-off — weaker evidence. Acceptable, but the limitation must be stated in any compliance claim.	Phase 1
3	**What is the real production data**	CL-B-05 (restore RTO) is	Phase 4

	volume?	meaningless at 10 GB if production is 5 TB. DR RTO cannot be validated without this figure.	
4	**Are the current SLA targets derived or aspirational?**	RTO ≤ 60 s, RPO 0 and failover ≤ 30 s appear to be stated rather than measured. The recommendation is to measure first, then commit.	Phase 3
5	**Which environment classes are contractually supported at launch?**	E1–E4 is the assumed answer, but each class we support requires lab representation and multiplies the 21 environment-sensitive scenarios. Dropping E4 (burstable) would be defensible — credit cliffs make any RTO guarantee fragile — and would	Phase 1

		reduce both lab cost and support risk.	
6	**Which storage classes will we certify for RPO = 0?**	Durability is a property of storage, not compute. Each certified class (local NVMe, BBU RAID, SAN, network block, NFS) needs its own EQS-D run and lab representation. NFS in particular is high-risk and may warrant explicit exclusion.	Phase 1
7	**Is the Environment Qualification Suite a shipped product deliverable or an internal/field-engineer tool?**	If customer-runnable it needs packaging, documentation, safety rails (EQS-D destroys the instance under test) and a defensible report format. If field-only, it can stay internal. This materially changes Phase 2 scope.	Phase 2

8	**Is containerization on the roadmap, and when?**	Determines whether container-native fault-injection adapters should be built now or later, and whether E5 scenarios should be authored now or later.	Phase 2

20. Document Approval

Role	Name	Signature	Date
Chief Architect			
DistDB Engineering Lead			
Product Owner			
QA Lead			
Engineering Director			

Unified Resilience Testing Framework — ShaktiDB (PostgreSQL) and DistDB

Version 1.0 · Database Reliability Engineering · CONFIDENTIAL — INTERNAL USE ONLY

162 scenarios · 104 P0 · 3 tiers · 2 systems

Review quarterly, or after any significant infrastructure or architecture change.